

TITLE OF THE APPLICATION- FINANCIAL DOCUMENT TAMPERING DETECTION SYSTEM USING EFFICIENTNETV2B0 AND GRAD-CAM

A project report submitted in partial fulfillment of the requirements for the
completion of application development in

Bachelor of Technology

In

Information Technology

By

M. Udayini (2411IT010142)

C. Anjani (2411IT010063)

C. Sai Kiran (2411IT010067)

Under the guidance of

Mrs. G. Devi Priya

Assistant Professor



Department of Information Technology

MALLA REDDY UNIVERSITY, HYDERABAD

2026-2027



MALLA REDDY UNIVERSITY

(Telangana State Private Universities Act No.13 of 2020 and G.O.Ms.No.14, Higher Education (UE) Department)

CERTIFICATE

This is to certify that this is the Application development lab record entitled “FINANCIAL DOCUMENT TAMPERING DETECTION SYSTEM USING EFFICIENTNETV2B0 AND GRAD-CAM”, submitted by M. UDAYINI (2411IT010142), C. ANJANI (2411IT010063), C. SAI KIRAN (2411IT010067), B. Tech III year I semester, Department of IT during the year 2026-27. The results embodied in this report have not been submitted to any other university or institute for the award of any degree or diploma.

Internal Guide
Mrs. G. Devi Priya

HOD IT
Dr. Arun Singh Chouhan

External Examiner



MALLA REDDY UNIVERSITY

(Telangana State Private Universities Act No.13 of 2020 and G.O.Ms.No.14, Higher Education (UE) Department)

DECLARATION

We declare that this project report titled FINANCIAL DOCUMENT TAMPERING DETECTION SYSTEM USING EFFICIENTNETV2B0 AND GRAD-CAM, submitted in partial fulfillment of the degree of B. Tech in IT is a record of original work carried out by us under the supervision of Mrs. G. Devi Priya, and has not formed the basis for the award of any other degree or diploma, in this or any other Institution or University. In keeping with the ethical practice in reporting scientific information, due acknowledgements have been made wherever the findings of others have been cited.

M. UDAYINI	2411IT010142
C. ANJANI	2411IT010063
C. SAI KIRAN	2411IT010067

ACKNOWLEDGEMENT

It gives me immense pleasure to express my profound gratitude to all those who extended their valuable support, encouragement, and guidance in the successful completion of this project.

First and foremost, we would like to express my heartfelt thanks to Dr. V. S. K. Reddy, Hon'ble Vice-Chancellor of our esteemed institution, for providing an excellent academic environment, necessary facilities, and constant encouragement that motivated me to pursue this project with enthusiasm and dedication. His inspiring leadership has always been a source of motivation for all students.

We deeply indebted to Dr. Arun Singh Chouhan, HOD, Department of Information Technology, for his valuable support, expert advice, and continuous encouragement throughout the course of this project. His guidance and concern for students have greatly contributed to the successful completion of this work.

We wish to express my sincere appreciation and heartfelt gratitude to our project guide, MRS. G. DEVI PRIYA, for their invaluable guidance, constructive suggestions, and constant supervision during every phase of this project. Their technical expertise, insightful feedback, and encouragement were instrumental in shaping this work and bringing it to completion.

We also extend my thanks to all the faculty members of the Department of Information Technology for their cooperation and assistance whenever required.

Finally, we express my deep gratitude to my parents, friends, and well-wishers for their unwavering support, patience, and motivation throughout this journey. Their encouragement has been the driving force behind the successful completion of this project.

M. UDAYINI	2411IT010142
C. ANJANI	2411IT010063
C. SAI KIRAN	2411IT010067



MALLA REDDY UNIVERSITY

(Telangana State Private Universities Act No.13 of 2020 and G.O.Ms.No.14, Higher Education (UE) Department)

INFORMATION TECHNOLOGY

ABSTRACT

Financial document tampering has become a serious challenge for banks, insurance companies and business organizations due to the increasing use of digitally manipulated invoices, receipts, bills and financial records. Manual verification of these documents is time-consuming, error-prone and inefficient. This project presents FinDoc AI, an explainable financial document tampering detection system based on EfficientNetV2B0 and Gradient-weighted Class Activation Mapping (Grad-CAM). The system is trained on the public "Find it again!" receipt forgery dataset of 988 scanned receipts with separate training, validation and testing splits. EfficientNetV2B0, initialised with ImageNet weights, is fine-tuned on 224 x 224 windows cut from the receipts at their original resolution, and complete documents are analysed by scanning them with overlapping windows and reporting the most suspicious region. Grad-CAM heatmaps, outlined suspicious regions and a plain-language list of reasons explain every decision. The interactive web application, developed with Flask, HTML, CSS and JavaScript, accepts images, camera photos, PDF, Word and PowerPoint files, and assesses typed digital files through their metadata. A document-level evaluation revealed that an initial model with 97.85% reported patch accuracy had learned the size of its training patches instead of tampering and performed at chance level on complete receipts; after correcting the patch preparation, the final system reaches 43.58% accuracy and a document-level AUC of 47.04% on 218 test receipts, detecting 22 of 35 tampered receipts. The results show that detecting careful manual edits in scanned receipts remains difficult, and the system is best used as an explainable screening aid that shows a human reviewer where to look.

Keywords: Document Forgery Detection, Financial Fraud, Deep Learning, EfficientNetV2B0, Transfer Learning, Explainable AI (XAI), Grad-CAM, Flask.



MALLA REDDY UNIVERSITY

(Telangana State Private Universities Act No.13 of 2020 and G.O.Ms.No.14, Higher Education (UE) Department)

INFORMATION TECHNOLOGY

CONTENTS

Certificate		II
Declaration		III
Acknowledgement		IV
Abstract		V
Contents		VI
List of Figures		VII
List of Tables		VIII
Abbreviations		IX
Chapter 1	Introduction	1
	1.1 Introduction	1
	1.2 Problem Statement	3
	1.3 Objective of Project	4
	1.4 Goal of Project	4
Chapter 2	Problem Identification	6
	2.1 Existing System	6
	2.2 Proposed System	10
Chapter 3	Requirements	11
	3.1 Software Requirements	11
	3.2 Hardware Requirements	12
Chapter 4	Design and Implementation	13
	4.1 Design	13
	4.2 Implementation	19
Chapter 5	Code	28
	5.1 Source Code	28
	5.2 Screenshot of the Application	34
Chapter 6	Conclusion	40
	6.1 References	47
	6.2 Appendix : Source Code	49



MALLA REDDY UNIVERSITY

(Telangana State Private Universities Act No.13 of 2020 and G.O.Ms.No.14, Higher Education (UE) Department)

INFORMATION TECHNOLOGY

LIST OF FIGURES

Figure No.	Title	Page
4.1	System Architecture of FinDoc AI	14
4.2	Level 0 Data Flow Diagram (Context Diagram)	15
4.3	Level 1 Data Flow Diagram	15
4.4	Workflow of the Tampering Detection Process	16
4.5	Use Case Diagram	17
4.6	Sequence Diagram for Document Analysis	18
4.7	Class Diagram	19
4.8	Deployment Diagram	19
4.9	Five-step Analysis Pipeline of FinDoc AI	21
4.10	EfficientNetV2B0 based Tampering Classification Model	23
4.11	Grad-CAM Heatmap Generation Process	25
5.1	Home Page of FinDoc AI	34
5.2	Detect Page – Upload a File or Use the Camera	35
5.3	Result Page for a Tampered Receipt – Verdict and Reasons	36
5.4	Result Page – Original Document and Full-page Grad-CAM Map	37
5.5	Result Page – Zoomed Suspicious Region and Analysis Steps	37
5.6	Result Page for a Genuine Receipt	38
5.7	Result Page for an Edited Digital PDF (Metadata Check)	38
5.8	Performance Page Showing the Measured Test Results	39
6.1	Training and Validation Accuracy and Loss Curves	41
6.2	Document-level Evaluation Metrics	42
6.3	Confusion Matrix on the Test Receipts	42
6.4	ROC Curve on the Test Receipts	42



MALLA REDDY UNIVERSITY

(Telangana State Private Universities Act No.13 of 2020 and G.O.Ms.No.14, Higher Education (UE) Department)

INFORMATION TECHNOLOGY

LIST OF TABLES

Table No.	Title	Page
2.1	Summary of Literature Survey	8
2.2	Types of Tampering and their Visual Traces	9
3.1	Software Requirements	11
3.2	Python Libraries and their Purpose	11
4.1	Documents and Edited Regions in the Dataset	20
4.2	Number of 224×224 Training Patches	21
4.3	Comparison of Pre-trained CNN Backbones (Keras Applications, ImageNet)	23
4.4	Training Configuration and Hyperparameters	24
6.1	Document-level Results on the Test Receipts	41
6.2	Initial Model versus Final Model	43
6.3	Functional Test Cases	45



MALLA REDDY UNIVERSITY

(Telangana State Private Universities Act No.13 of 2020 and G.O.Ms.No.14, Higher Education (UE) Department)

INFORMATION TECHNOLOGY

ABBREVIATIONS

Abbreviation	Full Form
AI	Artificial Intelligence
AUC	Area Under the ROC Curve
CAM	Class Activation Mapping
CNN	Convolutional Neural Network
CPU / GPU	Central / Graphics Processing Unit
CSS	Cascading Style Sheets
DFD	Data Flow Diagram
DL	Deep Learning
ELA	Error Level Analysis
FN / FP	False Negative / False Positive
Grad-CAM	Gradient-weighted Class Activation Mapping
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
JPEG	Joint Photographic Experts Group
MBConv	Mobile Inverted Bottleneck Convolution
ML	Machine Learning
OCR	Optical Character Recognition
PDF	Portable Document Format
ReLU	Rectified Linear Unit
ROC	Receiver Operating Characteristic
SE	Squeeze-and-Excitation
TN / TP	True Negative / True Positive
UI	User Interface
UML	Unified Modeling Language
XAI	Explainable Artificial Intelligence

CHAPTER-1

INTRODUCTION

1.1 INTRODUCTION

Financial documents such as invoices, receipts, bank statements, salary slips and bills are the backbone of everyday business and banking operations. They are used as evidence for expense reimbursement, loan processing, insurance claims, tax filing and audit trails. With the rapid shift to digital workflows, most of these documents are now exchanged as scanned images or photographs taken on mobile phones. While this digitization has made financial processes faster, it has also made document manipulation remarkably easy. Freely available image editing tools allow anyone to change an amount, a date, a merchant name or a line item on a receipt within minutes, and the result is often visually indistinguishable from the original.

Document fraud of this kind causes significant financial losses to organizations. Inflated expense claims, forged invoices submitted to accounts-payable teams and manipulated supporting documents in loan or insurance applications are common examples. Traditionally, such documents are verified manually by an employee who inspects each one for inconsistencies. Manual verification is slow, depends heavily on the expertise and attention of the reviewer, and does not scale to the thousands of documents that a medium-sized organization processes every day. Subtle digital edits, such as a single copied digit or a re-typed amount in a matching font, are almost impossible to detect by eye.

Deep learning has transformed the field of computer vision over the last decade. Convolutional Neural Networks (CNNs) can learn to recognize very fine visual patterns, including the traces left behind by digital editing: inconsistent compression artifacts, unnatural edges around pasted regions, mismatched font rendering and local noise inconsistencies. However, a model that only outputs a “tampered” label is of limited use to a human auditor, who needs to know where the manipulation is and why the model reached its decision. This is where Explainable Artificial Intelligence (XAI) becomes important.

This project, FinDoc AI – Financial Document Tampering Detection System, proposes an explainable deep learning solution that automatically classifies a financial document image as Genuine or Tampered. The system uses EfficientNetV2B0, a modern and highly efficient CNN architecture, fine-tuned through transfer learning on a public financial document forgery dataset. To make every prediction transparent, the system applies Grad-CAM (Gradient-weighted Class Activation Mapping), which produces a colour heatmap over the document that highlights the regions that most influenced the decision. The complete pipeline is delivered as an interactive web application built with Flask, HTML, CSS and JavaScript, in which a user can upload a document or capture it with the device camera and see the result instantly, without creating an account.

1.1.1 Convolutional Neural Networks

A Convolutional Neural Network (CNN) is a deep learning model designed for grid-structured data such as images. Instead of connecting every pixel to every neuron, a CNN slides small learnable filters (kernels) across the image. Each filter responds to a particular local pattern – an edge, a corner, a stroke of a character or a texture – and produces a feature map that shows where that pattern occurs. Stacking many convolutional layers allows the network to build a hierarchy of features: early layers detect simple edges and noise patterns, middle layers detect shapes and character strokes, and deep layers detect complex structures such as text blocks, table lines and logos.

A typical CNN combines four kinds of layers. Convolution layers extract features using shared weights, which makes the model efficient and translation invariant. Activation functions such as ReLU or SiLU introduce non-linearity so that the network can learn complex decision boundaries. Pooling layers reduce the spatial size of feature maps, making the representation compact and robust to small shifts. Finally, fully connected (dense) layers combine the extracted features to produce the output, which in this project is a single probability that the document is tampered.

1.1.2 Transfer Learning

Training a deep CNN from scratch requires millions of labelled images, but public document forgery datasets contain only a few thousand images. Transfer learning solves this problem by starting from a network that has already been trained on a large general dataset – ImageNet, with 1.28 million images in 1000 classes [16] – and adapting it to the new task. The pre-trained filters already know how to detect edges, textures and shapes, so only the higher layers need to be adjusted to recognize manipulation traces.

1.1.3 Explainable Artificial Intelligence

Explainable Artificial Intelligence (XAI) refers to methods that make the decisions of machine learning models understandable to humans. Deep networks contain millions of parameters and are often described as black boxes. In sensitive domains such as finance, healthcare and law, a prediction without a reason is difficult to accept, because the person who acts on the prediction remains responsible for the decision. XAI methods provide this reason, either globally (how the model behaves in general) or locally (why it made a particular prediction).

1.2 PROBLEM STATEMENT

Organizations receive large volumes of financial documents in image form, and a growing fraction of these documents are digitally manipulated to alter amounts, dates or other critical fields. Manual verification is time-consuming, error-prone and cannot reliably detect subtle digital edits. Existing automated approaches either rely on hand-crafted forensic features that fail on real-world documents, or use deep learning models that behave as black boxes and give no explanation for their decisions. There is a need for an accurate, efficient and explainable system that can automatically classify a financial document image as genuine or tampered, visually indicate the suspicious regions, and make this capability available to non-technical users through a simple web interface.

1.3 OBJECTIVE OF PROJECT

The main objectives of the project are:

To study the problem of financial document tampering and review existing forgery detection techniques.

To prepare a financial document image dataset with separate training, validation and test splits and to cut training patches that keep the original resolution of the receipts.

To build and fine-tune an EfficientNetV2B0 model through transfer learning for binary classification of documents as Genuine or Tampered.

To integrate Grad-CAM so that every prediction is accompanied by a heatmap that highlights the suspicious regions of the document.

To evaluate the model using standard metrics such as accuracy, precision, recall, F1-score and ROC-AUC.

To develop an interactive, responsive web application using Flask, HTML, CSS and JavaScript that supports document upload and camera capture, result visualization with Grad-CAM, and pages that explain the pipeline and report the model performance.

1.4 GOAL OF PROJECT

The goal of the project is to deliver a working, explainable decision-support tool that helps finance teams, auditors and banking staff screen financial documents for manipulation. Rather than replacing the human reviewer, FinDoc AI aims to reduce the reviewer's workload by automatically flagging suspicious documents and showing exactly where the suspicious evidence lies, so that verification becomes faster, more consistent and more transparent.

1.4.1 Key Features

Automatic tampering detection – classifies an uploaded invoice or receipt image as Genuine or Tampered within seconds.

Tampering score and threshold – displays the score of the most suspicious region next to the decision threshold, so the user can see how clear-cut the result is.

Explainable AI with Grad-CAM – overlays a heatmap on the document to highlight the regions that influenced the decision.

Efficient deep learning backbone – EfficientNetV2B0 offers high accuracy with a small number of parameters (about 7.2 million), enabling fast inference even on a CPU.

No sign-up required – an interactive web application: users open the site, add a document and receive the result immediately; no personal data is stored.

Performance transparency – a dedicated Performance page displays the accuracy, precision, recall, AUC and loss of the trained model.

Camera capture – documents can be uploaded as image files or captured directly with the device camera.

Region-based analysis – large documents are divided into smaller regions that are analyzed separately and combined into a document-level decision.

Modern responsive interface – Home, Detect, How It Works, Performance and About pages built with HTML, CSS and JavaScript.

CHAPTER-2

PROBLEM IDENTIFICATION

2.1 EXISTING SYSTEM

In most organizations today, financial documents are verified manually. An employee in the finance, audit or loan-processing team visually inspects each submitted invoice or receipt, cross-checks totals and dates, and sometimes contacts the vendor for confirmation. Some organizations supplement this with rule-based checks in their expense software, such as duplicate-receipt detection or limits on claim amounts, and with OCR-based extraction that compares the extracted values with the values entered by the claimant. In research settings, forgery detection is performed with classical forensic techniques such as ELA and noise analysis, or with deep learning classifiers that output only a class label.

2.1.1 Literature Survey

Image forgery detection has been studied for more than two decades. Early research in digital image forensics relied on hand-crafted statistical features. Error Level Analysis (ELA), popularized by Krawetz [13], re-compresses a JPEG image at a known quality and highlights regions whose compression error differs from the rest of the image, which often indicates pasted content. Fridrich and Kodovsky [12] proposed rich models built from a large set of noise residual filters, which were later adapted for splicing and copy-move detection. These techniques are simple and interpretable, but they are sensitive to recompression, resizing and scanning, all of which are very common for real financial documents.

Specifically for documents, Artaud et al. [8] organized the “Find it!” fraud detection contest at ICPR 2018, which introduced a dataset of real receipts with realistic forgeries made by volunteers and showed that detecting such forgeries is much harder than detecting forgeries in natural photographs. Cruz et al. [22] used local binary pattern texture descriptors to detect forged regions in documents. More recently, Tornés et al. [7] released a receipt dataset for document forgery detection with genuine and tampered receipts and fixed training, validation and test splits,

and reported that standard deep learning baselines still leave considerable room for improvement. These datasets established document tampering as a distinct research problem.

The success of deep learning in image classification, starting with AlexNet [6] and continuing with VGG [5] and ResNet [4], encouraged researchers to apply CNNs to forensics. Bayar and Stamm [11] introduced a constrained convolutional layer that learns to suppress image content and focus on manipulation traces. Zhou et al. [9] proposed a two-stream Faster R-CNN that combines RGB features with noise features to localize tampered regions, and Wu et al. [10] proposed ManTra-Net, which learns manipulation traces for many types of edits. Harley et al. [21] showed that CNNs pre-trained on ImageNet [16] transfer very well to document images, which supports the use of transfer learning for document tasks.

EfficientNet [2] introduced compound scaling of depth, width and resolution and achieved state-of-the-art accuracy with far fewer parameters than earlier CNNs. EfficientNetV2 [1] improved training speed and parameter efficiency further by using Fused-MBConv blocks in the early stages and training-aware neural architecture search. The smallest variant, EfficientNetV2B0, is especially suitable for web deployment because it combines strong accuracy with low computational cost.

To address the black-box nature of CNNs, Zhou et al. [17] proposed Class Activation Mapping (CAM), which requires a specific network structure. Selvaraju et al. [3] generalized this idea with Grad-CAM, which uses the gradients of the target class flowing into the last convolutional layer to produce a coarse localization map for any CNN without architectural changes. Grad-CAM++ [18] later improved localization when multiple regions are involved. These XAI techniques have been widely adopted in medical imaging and are increasingly used in forensic applications, where a visual justification of the decision is essential. Table 2.1 summarizes the key works reviewed.

Table 2.1: Summary of Literature Survey

Ref.	Author(s) & Year	Technique	Key Observation / Limitation
[13]	Krawetz, 2007	Error Level Analysis	Simple and interpretable; unreliable after recompression or scanning.
[12]	Fridrich & Kodovsky, 2012	Rich noise-residual models	Strong hand-crafted features; high dimensional and dataset specific.
[11]	Bayar & Stamm, 2016	Constrained CNN layer	Learns manipulation traces automatically; designed for natural images.
[9]	Zhou et al., 2018	Two-stream Faster R-CNN (RGB + noise)	Localizes tampered regions; heavy model, needs pixel-level labels.
[8]	Artaud et al., 2018	Find it! receipt fraud contest	Shows realistic document forgeries are very hard to detect.
[10]	Wu et al., 2019	ManTra-Net	Generic manipulation tracing; not tuned for text documents.
[2], [1]	Tan & Le, 2019 / 2021	EfficientNet / EfficientNetV2	High accuracy with few parameters; fast training and inference.
[3]	Selvaraju et al., 2017	Grad-CAM	Visual explanation for any CNN without changing the architecture.
[7]	Tornés et al., 2023	Receipt forgery dataset + baselines	Public benchmark with train/val/test splits; room for improvement.

2.1.2 Types of Financial Document Tampering

Financial documents can be manipulated in several ways. Understanding these manipulation types helps in designing a detection system and in interpreting its explanations. The most common types are described below and summarized in Table 2.2.

Copy-move forgery copies a region of the same document – typically a digit or a word – and pastes it elsewhere, for example to turn ₹4,500 into ₹8,500 by copying an “8” from another amount. Because the pasted content comes from the same image, its colour and font match perfectly, which makes it very hard to notice.

Splicing pastes content taken from a different document, such as a logo, stamp or signature, and often leaves differences in resolution, noise or compression.

Text insertion adds new characters or line items using an editor with a font that imitates the original print. Erasure removes information by painting over it with the background colour or by using a healing tool, for instance to hide a discount or a cancelled item. Re-typing combines erasure and insertion: the original date or amount is removed and a new value is typed in its place. In all cases the edited region carries subtle traces – sharper or blurrier edges, missing thermal-printer noise, misaligned baselines or different JPEG block artifacts – that a CNN can learn to detect.

Table 2.2: Types of Tampering and their Visual Traces

Tampering Type	Description	Typical Visual Traces
Copy-move	Region copied and pasted within the same document	Duplicated patterns, abrupt edges, repeated noise
Splicing	Content pasted from another document	Different resolution, noise level or compression
Text insertion	New text typed with a similar font	Font or spacing mismatch, too-clean characters
Erasure	Content removed by painting over it	Smooth patches, missing texture or print noise
Re-typing	Original value erased and new value typed	Combination of erasure and insertion traces

2.1.3 Limitations of Existing System

Manual verification is slow and depends on human expertise; it does not scale to large volumes of documents.

Human reviewers cannot see pixel-level edits, such as a single digit copied from another part of the same receipt.

Rule-based and OCR checks only verify consistency of the text; a consistently edited document passes all such checks.

Many deep learning models act as black boxes and do not explain their predictions, which makes auditors reluctant to trust them.

2.2 PROPOSED SYSTEM

The proposed system, FinDoc AI, is an explainable deep learning web application for detecting tampering in financial document images. The user uploads a receipt or invoice as an image, PDF or Word file, or captures it with the device camera. The Flask backend converts the file to page images, converts each page to grey-scale and scans it with overlapping 224×224 windows at its original resolution, so that edits of only a few pixels are not lost by resizing. Every window is scored by an EfficientNetV2B0 model that has been pre-trained on ImageNet and fine-tuned on the public Find it again! receipt forgery dataset, and the document is reported as tampered when its most suspicious window reaches a decision threshold chosen on validation documents.

For every prediction, the system computes Grad-CAM heatmaps for the most suspicious windows and overlays them on the full page, outlines the flagged regions, shows the most suspicious region enlarged and lists the reasons for the decision in plain language. The application is fully interactive and requires no login, so it can be used immediately by anyone who needs to check a document.

2.2.1 Advantages of Proposed System

Explainable – the Grad-CAM heatmap shows where the suspicious evidence is, which builds trust and speeds up manual review.

Low cost – transfer learning with the compact EfficientNetV2B0 model trains and runs on an ordinary laptop without a GPU.

CHAPTER-3

REQUIREMENTS

This chapter describes the functional and non-functional requirements of FinDoc AI, followed by the software and hardware required to develop and run the system.

3.1 SOFTWARE REQUIREMENTS

Table 3.1: Software Requirements

Component	Specification
Operating System	Windows 10 / 11 (also runs on Linux or macOS)
Programming Language	Python 3.10 or above
Deep Learning Framework	TensorFlow 2.20 with Keras 3.12
Image and Document Processing	Pillow, NumPy, PyMuPDF (PDF), Microsoft Word and PowerPoint (DOC, DOCX, PPT, PPTX)
Web Framework (Backend)	Flask 3.1
Frontend	HTML5, CSS3, JavaScript (camera capture with getUserMedia)
IDE / Tools	Visual Studio Code, Python virtual environment (venv)
Browser	Google Chrome, Microsoft Edge or Mozilla Firefox

3.1.1 Python Libraries Used

Table 3.2: Python Libraries and their Purpose

Library	Purpose in the Project
tensorflow / keras	Building, training and loading the EfficientNetV2B0 model; computing gradients for Grad-CAM
pillow	Reading images, grey-scale conversion, cropping patches and windows
pymupdf / pywin32	Rendering PDF pages and reading PDF metadata; exporting Word and PowerPoint files with Office
scikit-learn	Class weights and evaluation metrics
matplotlib	Grad-CAM overlays, training curves, confusion matrix and ROC curve
flask	Web server, routing and templates

3.1.2 Functional Requirements

The system shall allow the user to upload an image, PDF, Word or PowerPoint file, or to capture the document with the device camera.

The system shall assess typed digital documents through their metadata instead of the image model.

The system shall validate the file type and size and reject invalid files with a clear message.

The system shall preprocess the uploaded image and classify it as Genuine or Tampered using the trained EfficientNetV2B0 model.

The system shall display the prediction label, the tampering score, the decision threshold and the reasons for the decision.

The system shall generate and display a Grad-CAM heatmap overlaid on the document.

The system shall divide large documents into regions and combine the regional scores into a document-level decision.

3.2 HARDWARE REQUIREMENTS

- Processor: Intel Core i5 or above (or an equivalent AMD processor)
- RAM: 8 GB (16 GB recommended for training)
- Storage: 10 GB of free disk space
- GPU (optional, for training): NVIDIA GPU with CUDA support, or Google Colab GPU
- Camera: device camera or webcam (optional, for capturing documents)
- Network: Internet connection (to download libraries, pre-trained weights and the dataset)

CHAPTER-4

DESIGN AND IMPLEMENTATION

This chapter presents the design of FinDoc AI – its architecture, data flow, workflow and UML diagrams – followed by the implementation details of the dataset preparation, preprocessing, region-based analysis, model, training, explainability and application modules.

4.1 DESIGN

4.1.1 System Architecture

FinDoc AI is an interactive web application: the user opens the website, adds a document and receives the analysis immediately, without creating an account. The system follows a three-layer architecture, as shown in Figure 4.1. The presentation layer consists of the web pages that the user interacts with: the Detect page (upload or camera capture), the Home page, the How It Works page, the Performance and About pages, and the Result view with the Grad-CAM visualization. The application layer is the Flask backend, which contains the upload and file-conversion module, the preprocessing module, the region division module, the prediction service and the Grad-CAM generator. The model and storage layer contains the fine-tuned EfficientNetV2B0 model file, which is loaded once when the server starts, and a file store that temporarily holds the uploaded image and the generated heatmap so that they can be displayed on the result page.

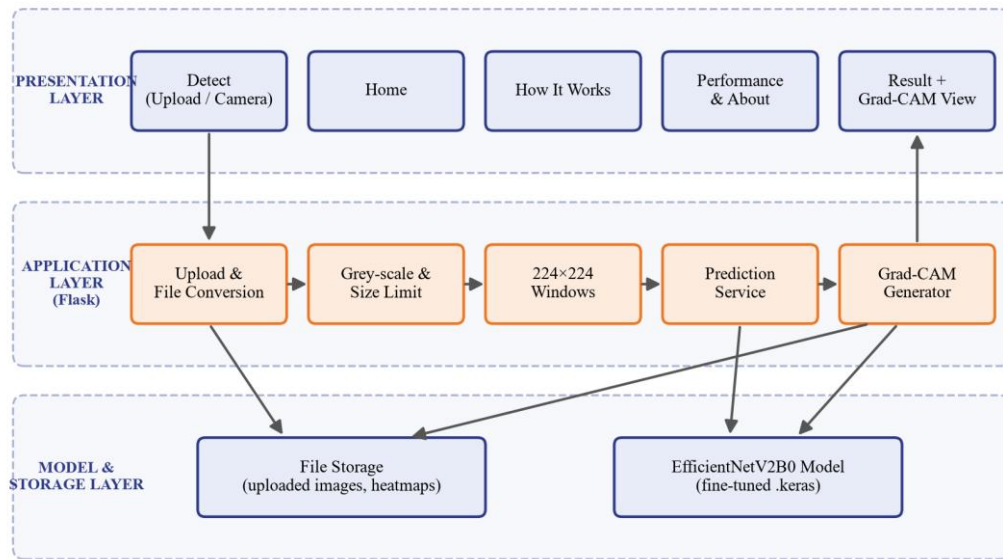


Figure 4.1: System Architecture of FinDoc AI

Separating the system into layers keeps each part independent. The user interface can be redesigned without touching the model, and the model can be retrained and replaced by simply swapping the .keras file. Because no user accounts or prediction records are kept, the system does not store any personal information, which keeps it simple, fast and privacy-friendly.

4.1.2 Data Flow Diagrams

Data Flow Diagrams (DFDs) show how data moves through the system. The Level 0 DFD, also called the context diagram (Figure 4.2), treats the entire system as a single process. The user sends a document image, either uploaded as a file or captured with the camera, and receives the prediction label, the tampering probability, the regional scores and the Grad-CAM heatmap. The system reads the trained model weights from the model file.

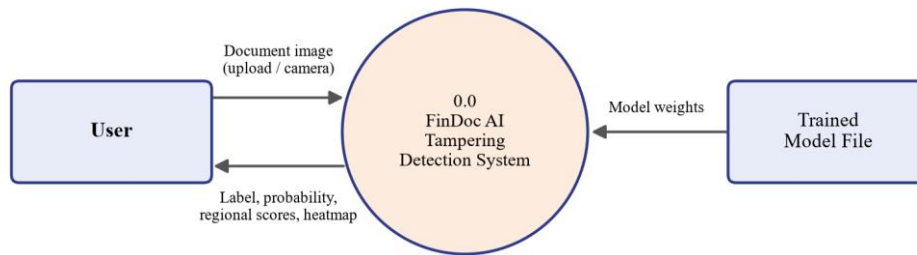


Figure 4.2: Level 0 Data Flow Diagram (Context Diagram)

The Level 1 DFD (Figure 4.3) breaks the system into its five main processes: (1.0) upload or capture and validate the document, (2.0) preprocess the image and divide it into regions, (3.0) classify each region with EfficientNetV2B0 using the stored model file, (4.0) generate the Grad-CAM heatmap and combine the regional scores, saving the images to file storage, and (5.0) display the result to the user.

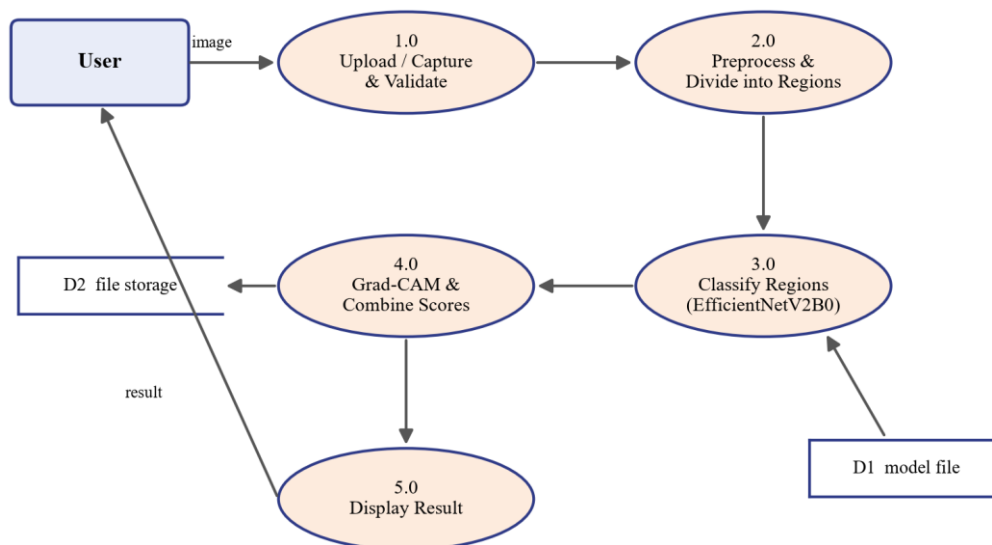


Figure 4.3: Level 1 Data Flow Diagram

4.1.3 System Workflow

Figure 4.4 shows the end-to-end workflow. The user opens the Detect page and either uploads a document image or captures one with the device camera. The system validates the file type and size; an invalid file returns the user to the Detect page with an error message. A valid document is converted to page images and to

grey-scale, and each page is divided into overlapping 224×224 regions. Each region is classified by the EfficientNetV2B0 model, Grad-CAM computes a heatmap, and the regional scores are combined into the final Genuine or Tampered decision. The result page shows the label, the tampering probability and the heatmap side by side with the original document.

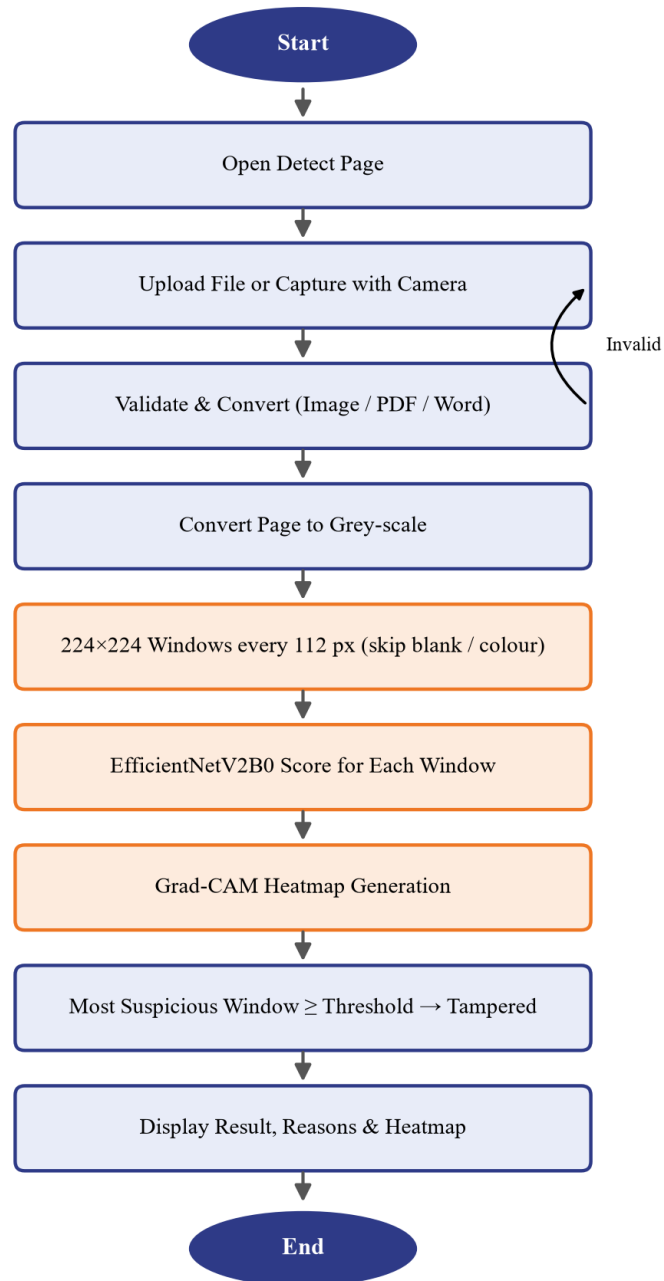


Figure 4.4: Workflow of the Tampering Detection Process

4.1.4 UML Diagrams

The Unified Modeling Language (UML) is used to describe the behaviour and structure of the system from different viewpoints. The use case, sequence, activity, class and deployment diagrams of FinDoc AI are presented below.

4.1.5 Use Case Diagram

The use case diagram in Figure 4.5 shows the interactions between the user and the system. The user can upload a document or capture one with the camera, view the prediction and tampering probability, view the Grad-CAM heatmap, learn how the system works, and view the model performance and About pages. The machine learning model participates in the upload, capture, prediction and heatmap use cases.

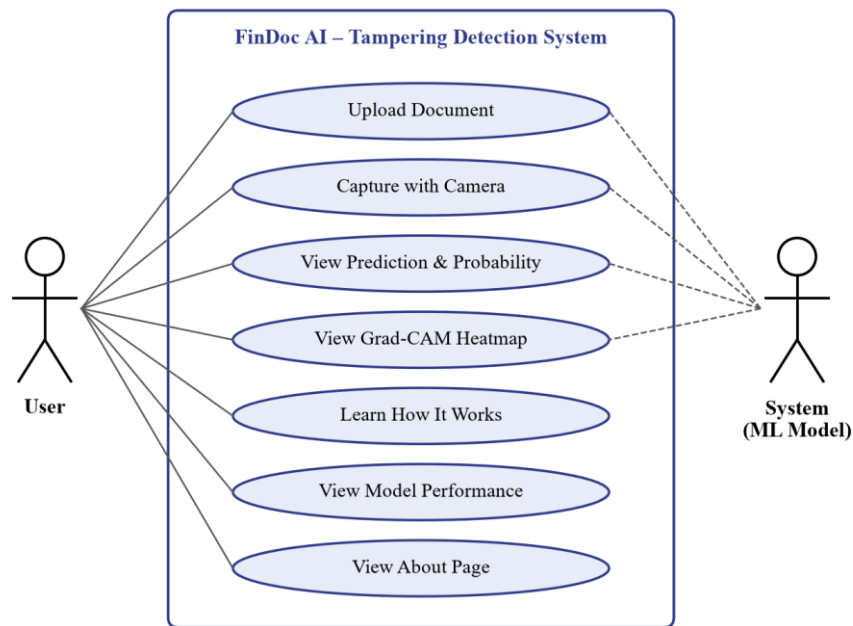


Figure 4.5: Use Case Diagram

4.1.6 Sequence Diagram

The sequence diagram in Figure 4.6 shows the order of messages exchanged when a document is analyzed. The browser sends the uploaded or captured image to the Flask server in a POST request. The server validates the file and asks the preprocessing module to prepare the image and divide it into 224×224 regions. It

then calls the prediction service, which runs the forward pass, computes the gradients for Grad-CAM and returns the regional scores and the heatmap. The server combines the scores into a label and renders the result page, which the browser displays to the user.

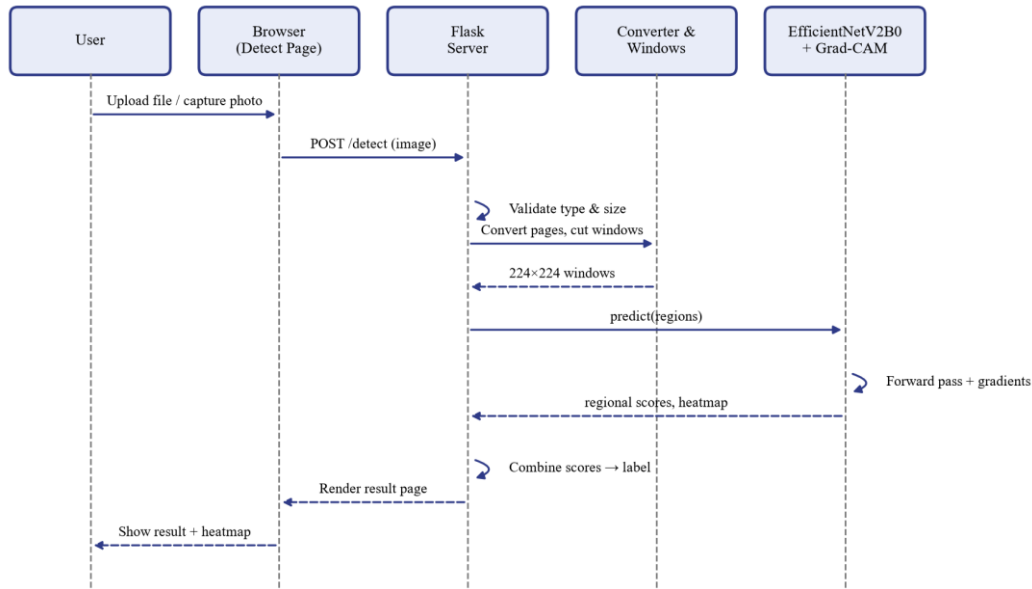


Figure 4.6: Sequence Diagram for Document Analysis

4.1.7 Class Diagram

The class diagram in Figure 4.7 shows the main software components. The FlaskApp class represents the routes of the web application and uses a single TamperDetector object, which holds the trained model and the Grad-CAM model and provides methods for regional scoring, prediction and heatmap generation. The TamperDetector relies on the Preprocessor, which prepares images and splits large documents into regions.

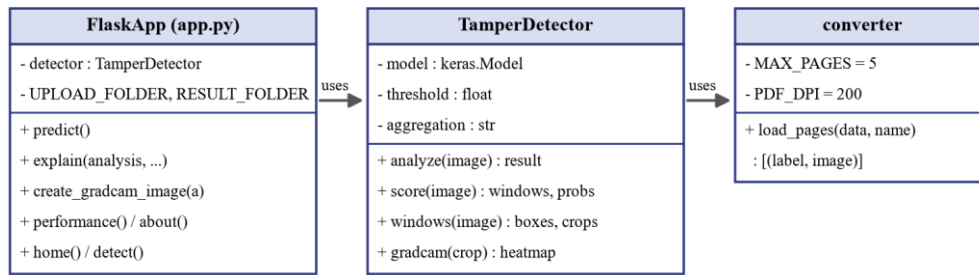


Figure 4.7: Class Diagram

4.1.8 Deployment Diagram

Figure 4.8 shows how the system is deployed. The client device – a PC or a mobile phone – runs only a web browser, and its camera can be used to capture documents. The application server runs Flask with TensorFlow, Pillow and PyMuPDF and holds the trained model file. The two communicate over HTTP. During development both run on the same laptop; the application server can later be hosted in the cloud without changing the client.

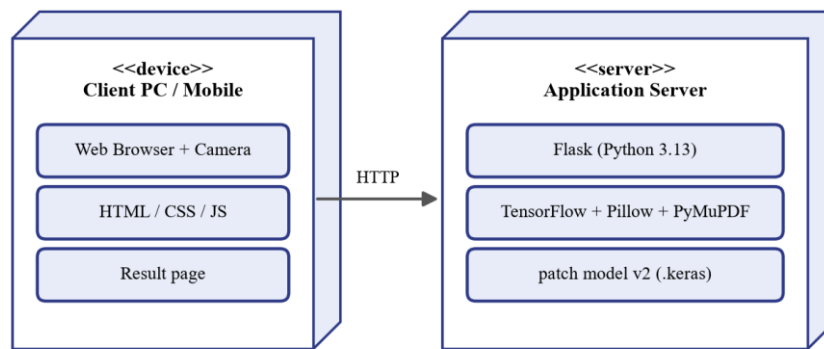


Figure 4.8: Deployment Diagram

4.2 IMPLEMENTATION

4.2.1 Dataset Description

The system is trained and evaluated on the public “Find it again!” receipt forgery dataset [7], which contains 988 scanned retail receipts. Forged receipts were created by volunteers who manually edited prices, totals, dates, product names and other

fields with ordinary image-editing software, and every edited area is annotated with a bounding box. The dataset is supplied with three annotation files (train.csv, val.csv and test.csv) that give each image, its label (forged = 1 or 0) and the coordinates of the edited regions, so that training, model selection and final evaluation use separate documents.

Table 4.1: Documents and Edited Regions in the Dataset

Split	Documents	Tampered documents	Edited regions
Training	577	94	273
Validation	193	34	91
Testing	218	35	101
Total	988	163	465

Two properties of the dataset make the task difficult. First, it is strongly imbalanced: only about one receipt in six is tampered. Second, the edits are very small – the median edited region is only about 37 pixels wide, often a single digit or character – and they were made carefully to be visually invisible. Balanced class weights are therefore used during training (Section 4.2.4), and the model is evaluated with precision, recall, F1-score and AUC in addition to accuracy (Chapter 6).

4.2.2 Image Preprocessing and Region-based Analysis

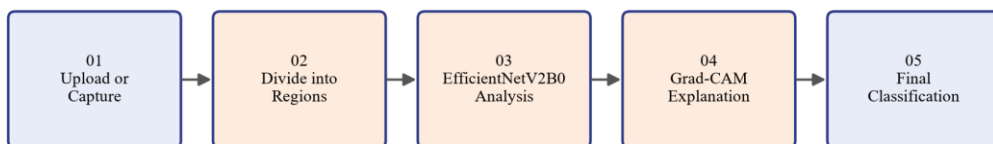
Patch preparation. The model does not look at a whole receipt at once, because a single 224×224 image of a long receipt would erase edits that are only a few pixels wide. Instead it is trained on 224×224 windows cut from the receipts at their original resolution (prepare_patches.py). For every annotated edit, six windows that fully contain the edited box are cut at random offsets and labelled Tampered. Genuine windows are cut at random positions from genuine receipts (five per receipt) and from the untouched parts of forged receipts (three per receipt), skipping blank paper. Table 4.2 lists the resulting patch counts.

Table 4.2: Number of 224×224 Training Patches

Split	Genuine patches	Tampered patches	Total
Training	2,695	1,638	4,333
Validation	510	180	690
Testing	584	202	786

Why the patch size matters. An earlier version of the project cut tampered patches tightly around the edit (about 80×85 pixels) but genuine patches at 256×256 pixels, and then resized both to 224×224 . The tampered crops were therefore enlarged about three times while the genuine crops were shrunk, and the network learned to recognise the zoom level instead of the tampering. It reported 97.85% patch accuracy, but on complete documents it performed no better than chance (Chapter 6). In the final version every patch, genuine or tampered, has the same size and scale, so the only difference the model can learn is the editing itself.

Document analysis. When a document is uploaded, the same scale is used: the page is converted to grey-scale (the training receipts are black-and-white scans), very large photographs are reduced to at most 2,400 pixels on the longer side, and the page is scanned with overlapping 224×224 windows every 112 pixels. Windows of blank paper are skipped, and so are strongly coloured windows such as logos or colour graphics, which never occur in the training receipts. Every remaining window is scored by the model, and the document score is the score of its most suspicious window. Figure 4.9 shows this five-step pipeline, which is also presented to users on the How It Works page.



Regional scores are combined into a document-level decision of Genuine or Tampered

Figure 4.9: Five-step Analysis Pipeline of FinDoc AI

No resizing, denoising or sharpening is applied to the windows, because the subtle pixel noise and edges around edited characters are exactly the traces that reveal tampering. For the same reason, zoom and rotation are not used for data augmentation; only horizontal flips and small brightness and contrast changes are applied.

4.2.3 EfficientNetV2B0 Model

EfficientNetV2 [1] is a family of convolutional networks designed by Tan and Le using training-aware neural architecture search and progressive scaling. Compared with the original EfficientNet, it trains faster and uses fewer parameters. Its key building blocks are:

Fused-MBConv blocks in the early stages, which replace the depthwise 3×3 convolution and 1×1 expansion with a single regular 3×3 convolution.

MBConv blocks (mobile inverted bottlenecks) in the later stages, which use an expansion convolution, a depthwise convolution and a projection convolution with a residual connection.

Squeeze-and-Excitation (SE) modules inside MBConv blocks, which re-weight channels according to their global importance.

Compound scaling of depth, width and resolution, of which B0 is the smallest baseline with about 7.2 million parameters.

EfficientNetV2B0 was chosen after comparing popular pre-trained backbones available in Keras Applications. As Table 4.3 shows, it offers the highest ImageNet accuracy among the compared models while being far smaller than VGG16 and ResNet50, which keeps training practical on a CPU and allows each document to be analysed in a few seconds.

Table 4.3: Comparison of Pre-trained CNN Backbones (Keras Applications, ImageNet)

Model	Parameters	Top-1 Accuracy	Remarks
VGG16	138.4 M	71.3 %	Very large; slow inference
ResNet50	25.6 M	74.9 %	Deep residual network; moderate size
MobileNetV2	3.5 M	71.3 %	Very small; lower accuracy
EfficientNetB0	5.3 M	77.1 %	Compound scaling; slower training
EfficientNetV2B0	7.2 M	78.7 %	Best accuracy–size trade-off; chosen

In this project the EfficientNetV2B0 backbone is loaded with ImageNet weights, without its original 1000-class classification layer. A custom head is added on top of the final $7 \times 7 \times 1280$ feature map: Global Average Pooling, Batch Normalization, Dropout (0.4), a Dense layer with 128 ReLU units, Dropout (0.3) and a single Dense neuron with sigmoid activation that outputs the probability that the window is tampered. Figure 4.10 shows the complete network.

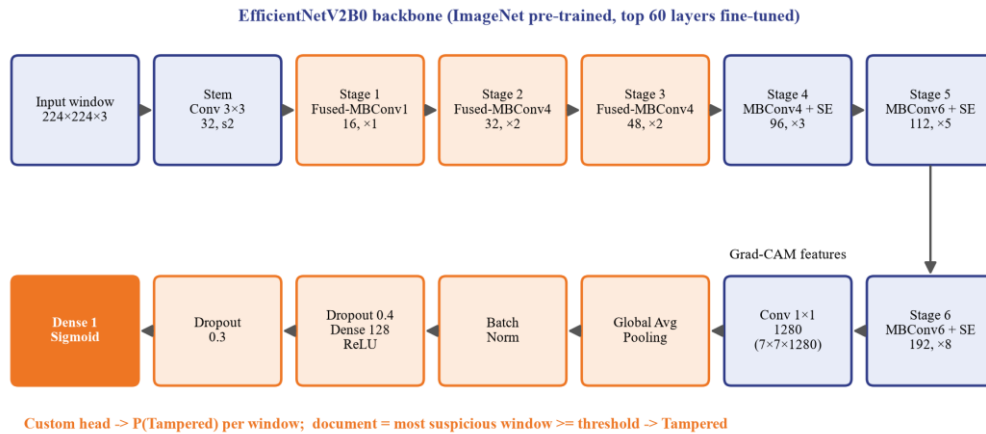


Figure 4.10: EfficientNetV2B0 based Tampering Classification Model

4.2.4 Model Training

Training (patch_train.py) uses transfer learning in two phases. In Phase 1 the backbone is frozen and only the new classification head is trained with a learning rate of 10^{-3} . In Phase 2 the top 60 layers of the backbone are unfrozen and fine-

tuned with a learning rate of 10^{-5} , while Batch Normalization layers stay frozen to keep their statistics stable on the small dataset. Balanced class weights (0.80 for genuine and 1.32 for tampered patches) compensate for the smaller number of tampered patches.

Early stopping monitors the validation AUC with a patience of five epochs and restores the best weights, and the learning rate is halved when the validation loss stops improving. Training ran for 4 epochs in Phase 1 and was stopped after 7 epochs of Phase 2, taking about 30 minutes on a laptop CPU. Table 4.4 lists the configuration.

Table 4.4: Training Configuration and Hyperparameters

Parameter	Value
Input	$224 \times 224 \times 3$ window at original resolution (no resizing)
Backbone	EfficientNetV2B0, ImageNet weights, include_top = False
Classification head	GAP $\rightarrow$ BatchNorm $\rightarrow$ Dropout 0.4 $\rightarrow$ Dense 128 (ReLU) $\rightarrow$ Dropout 0.3 $\rightarrow$ Dense 1 (sigmoid)
Augmentation	Horizontal flip, contrast $\pm 10\%$, brightness $\pm 8\%$ (no zoom or rotation)
Loss / optimizer	Binary cross-entropy with balanced class weights / Adam [19]
Learning rate	Phase 1: 1×10^{-3} ; Phase 2: 1×10^{-5} (top 60 layers)
Batch size / epochs	16 / Phase 1: 4, Phase 2: up to 16 (early stopping)
Callbacks	ModelCheckpoint and EarlyStopping on validation AUC, ReduceLROnPlateau
Document decision	Most suspicious window $\geq 77.85\%$ (threshold chosen on validation documents)

The decision threshold is not fixed at 50%. After training, evaluate.py scores every validation document and chooses the threshold that gives the best F1-score, together with the rule for combining the window scores (the maximum worked best). Both are stored in models/document_threshold.json and loaded by the web application.

4.2.5 Explainability using Grad-CAM

Grad-CAM [3] explains a CNN prediction by identifying which spatial regions of the last convolutional feature maps were most important for the predicted class. Let A^k be the k -th feature map produced by the EfficientNetV2B0 backbone ($7 \times 7 \times 1280$ for a 224×224 window) and let y^c be the tampering probability. The procedure, illustrated in Figure 4.11, is:

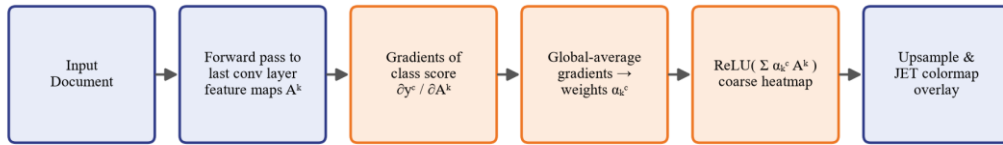
Step 1: Pass the window through the backbone and record the feature maps A^k .

Step 2: Pass A^k through the classification head inside a TensorFlow GradientTape and compute $\partial y^c / \partial A^k$.

Step 3: Global-average-pool the gradients to obtain one importance weight α_k^c for each feature map.

Step 4: Compute $L^c = \text{ReLU}(\sum_k \alpha_k^c A^k)$, which keeps only the areas that increase the tampering score.

Step 5: Normalise the map, up-sample it to 224×224 pixels and place it back at the window's position on the page.



Red/yellow regions = areas that most influenced the 'Tampered' decision

Figure 4.11: Grad-CAM Heatmap Generation Process

Grad-CAM is computed for the most suspicious windows of the document, weighted by their scores, combined into one full-page map, softened with a Gaussian blur so that window borders are not visible, and overlaid on the page with

the JET colour map. Red and yellow areas are the pixels that pushed the tampering score up the most; windows above the threshold are outlined in red.

4.2.6 Implementation Modules

Upload and Camera Capture Module

The Detect page offers Upload Document and Use Camera. The camera uses the browser's `getUserMedia` interface to capture a photo, which is sent to the server like an uploaded file. The backend (`converter.py`) accepts images (PNG, JPG, BMP, TIFF, WEBP), PDF files, Word documents and PowerPoint presentations up to 25 MB. PDF pages are rendered with PyMuPDF at 200 dpi (up to five pages). For Word and PowerPoint files, large pictures embedded in them (for example a pasted scan; logos and icons are ignored) are analysed directly; otherwise the file is exported to PDF with Microsoft Word or PowerPoint, opened read-only in a separate process with a time limit, so that the conversion also works when Office is not activated.

Preprocessing and Region Division Module

Implemented in `detector.py` with Pillow and NumPy. The `prepare` function limits very large images and pads small ones; the `windows` function converts the page to grey-scale, generates the overlapping 224×224 windows every 112 pixels and removes blank and strongly coloured windows.

Prediction Module

Implemented in the `TamperDetector` class. The model, the decision threshold and the score-combination rule are loaded once at start-up. For each page, all windows are scored in batches of 32, the document score is the highest window score, and the result is compared with the threshold. For multi-page files the page with the highest score is reported.

Grad-CAM Module

Uses the backbone and the classification-head layers of the loaded model to compute Grad-CAM for the most suspicious windows (Section 4.2.5). The full-page overlay and a zoomed view of the most suspicious region are saved as PNG files in static/results and shown on the result page.

Digital Document Metadata Module

Typed Word documents, PowerPoint presentations and PDF files with a text layer contain no scanning or pixel-level editing traces, so the image model cannot judge them; in testing it gave meaningless verdicts for such files. The application therefore assesses digital documents through their metadata (metadata.py).

For a PDF the module reads the creating program and PDF producer, the creation and modification dates and the number of times the file was saved (every incremental save adds an end-of-file marker), and it checks whether a known PDF-editing tool was used. For DOCX and PPTX files it reads the author, the person who last modified the file, the creation and modification dates, the revision count and the creating application. Each finding that indicates a change after creation – a modification date well after the creation date, several incremental saves of a PDF, an editing tool, or a last editor who differs from the author – is counted as an editing signal. The result page then reports “Signs of editing found” or “No signs of editing found” together with every finding. Because metadata can be removed or forged, the page states that the absence of signals is not proof that a document is genuine.

Frontend Module

HTML templates with a custom CSS stylesheet and JavaScript provide a responsive interface with five pages. Digital documents show the metadata assessment instead of the Grad-CAM map. The result page shows the verdict, the tampering score and threshold, region statistics, the original page and the Grad-CAM map, a zoomed view of the most suspicious region, a “Why this result” list of reasons and a “How it was decided” list of analysis steps.

CHAPTER-5

CODE

5.1 SOURCE CODE

The source code of the important modules of FinDoc AI is presented in this section: the model training module (patch_train.py) and the detection and Grad-CAM module (detector.py). The main part of the Flask application (app.py) is given in the Appendix (Section 6.2). The project is organized as follows:

```
Financial_Document_Tampering/
├── app.py                # Flask web application (routes, reasons)
├── detector.py           # window scanning, prediction, Grad-CAM
├── converter.py          # images, PDF and Word -> page images
├── prepare_patches.py    # 224 x 224 training patches
├── patch_train.py        # EfficientNetV2B0 training
├── evaluate.py           # document-level evaluation, threshold
├── requirements.txt
├── annotations/          # train.csv, val.csv, test.csv
├── dataset/              # train/, val/, test/ receipt images
├── patches/              # generated genuine/ and tampered/ patches
├── models/               # trained model, threshold, metrics
├── reports/              # confusion matrix, ROC and training curves
├── static/               # css/, js/camera.js, uploads/, results/
└── templates/            # index, detect, result, how_it_works,
                           performance and about pages
```

5.1.1 Model Training Module (patch_train.py)

```
import json
import os

import numpy as np
import tensorflow as tf
from tensorflow.keras import layers, Model
from tensorflow.keras.applications import EfficientNetV2B0

# ---- SETTINGS ----

BASE_DIR = os.path.dirname(os.path.abspath(__file__))
PATCH_DIR = os.path.join(BASE_DIR, "patches")
MODEL_DIR = os.path.join(BASE_DIR, "models")
VERSION = "v2"
MODEL_PATH = os.path.join(MODEL_DIR,
f"efficientnetv2b0_patch_tampering_{VERSION}.keras")
HISTORY_PATH = os.path.join(MODEL_DIR, f"training_history_{VERSION}.json")

# Patches are already 224 x 224 crops at the document's original resolution,
# so they are fed to the network without any resizing.
IMAGE_SIZE = (224, 224)
BATCH_SIZE = 16
HEAD_EPOCHS = 4
FINE_TUNE_EPOCHS = 16
FINE_TUNE_LAYERS = 60
```

```

# ---- LOAD PATCH DATASETS ----

def load(split, shuffle):
    return tf.keras.utils.image_dataset_from_directory(
        os.path.join(PATCH_DIR, split),
        labels="inferred",
        label_mode="binary",
        class_names=["genuine", "tampered"],
        image_size=IMAGE_SIZE,
        batch_size=BATCH_SIZE,
        shuffle=shuffle,
        seed=42,
    )

print("\n=====")
print("LOADING PATCH DATASETS")
print("=====")

train_dataset = load("train", True)
val_dataset = load("val", False)
test_dataset = load("test", False)

AUTOTUNE = tf.data.AUTOTUNE
train_dataset = train_dataset.cache().prefetch(AUTOTUNE)
val_dataset = val_dataset.cache().prefetch(AUTOTUNE)
test_dataset = test_dataset.prefetch(AUTOTUNE)

# ---- DATA AUGMENTATION ----

# No zoom and no rotation: both resample the image and blur the fine pixel
# traces left by editing. Only photometric changes and flips are used.
data_augmentation = tf.keras.Sequential([
    layers.RandomFlip("horizontal"),
    layers.RandomContrast(0.10),
    layers.RandomBrightness(0.08),
], name="data_augmentation")

# ---- BUILD MODEL (same architecture as before) ----

base_model = EfficientNetV2B0(include_top=False, weights="imagenet",
                             input_shape=(IMAGE_SIZE[0], IMAGE_SIZE[1], 3))
base_model.trainable = False

inputs = layers.Input(shape=(IMAGE_SIZE[0], IMAGE_SIZE[1], 3))
x = data_augmentation(inputs)
x = base_model(x, training=False)
x = layers.GlobalAveragePooling2D()(x)
x = layers.BatchNormalization()(x)
x = layers.Dropout(0.4)(x)
x = layers.Dense(128, activation="relu")(x)
x = layers.Dropout(0.3)(x)
outputs = layers.Dense(1, activation="sigmoid")(x)
model = Model(inputs, outputs)

def compile_model(learning_rate):
    model.compile(
        optimizer=tf.keras.optimizers.Adam(learning_rate=learning_rate),
        loss="binary_crossentropy",
        metrics=[
            "accuracy",
            tf.keras.metrics.Precision(name="precision"),
            tf.keras.metrics.Recall(name="recall"),
            tf.keras.metrics.AUC(name="auc"),
        ],
    )

```

```

# ---- CLASS WEIGHTS (counted from the patch folders) ----

genuine_count = len(os.listdir(os.path.join(PATCH_DIR, "train", "genuine")))
tampered_count = len(os.listdir(os.path.join(PATCH_DIR, "train",
"tampered")))
total = genuine_count + tampered_count
class_weights = {0: total / (2 * genuine_count), 1: total / (2 *
tampered_count)}

print("\n=====")
print("CLASS WEIGHTS")
print("=====")
print(f"genuine={genuine_count} tampered={tampered_count} ->
{class_weights}")

# ---- CALLBACKS ----

os.makedirs(MODEL_DIR, exist_ok=True)

def callbacks():
    return [
        tf.keras.callbacks.ModelCheckpoint(MODEL_PATH, monitor="val_auc",
mode="max",
                                save_best_only=True, verbose=1),
        tf.keras.callbacks.EarlyStopping(monitor="val_auc", mode="max",
patience=5,
                                restore_best_weights=True,
verbose=1),
        tf.keras.callbacks.ReduceLROnPlateau(monitor="val_loss", factor=0.5,
patience=2,
                                min_lr=1e-7, verbose=1),
    ]

# ---- TRAIN - phase 1: classification head only ----

print("\n=====")
print("PHASE 1: TRAINING THE CLASSIFICATION HEAD")
print("=====")

compile_model(1e-3)
history_1 = model.fit(train_dataset, validation_data=val_dataset,
epochs=HEAD_EPOCHS,
                    class_weight=class_weights, callbacks=callbacks())

# ---- TRAIN - phase 2: fine-tune the top of EfficientNetV2B0 ----

print("\n=====")
print("PHASE 2: FINE-TUNING EFFICIENTNETV2B0")
print("=====")

base_model.trainable = True
for layer in base_model.layers[:-FINE_TUNE_LAYERS]:
    layer.trainable = False
for layer in base_model.layers:
    if isinstance(layer, layers.BatchNormalization):
        layer.trainable = False

compile_model(1e-5)
history_2 = model.fit(train_dataset, validation_data=val_dataset,
epochs=FINE_TUNE_EPOCHS,
                    class_weight=class_weights, callbacks=callbacks())

```

```
# ---- TEST (patch level) ----

print("\n=====")
print("PATCH-LEVEL TEST RESULTS")
print("=====")

results = model.evaluate(test_dataset, return_dict=True)
for name, value in results.items():
    print(f"{name}: {value:.4f}")

history = {k: [float(v) for v in history_1.history[k] +
               history_2.history.get(k, [])]
            for k in history_1.history}
history["phase_1_epochs"] = len(history_1.history["loss"])
history["patch_test"] = {k: float(v) for k, v in results.items()}
with open(HISTORY_PATH, "w") as f:
    json.dump(history, f, indent=2)

model.save(MODEL_PATH)
print(f"\nModel saved at:\n{MODEL_PATH}")
print("Run python evaluate.py for document-level results.")
```

5.1.2 Detection and Grad-CAM Module (detector.py)

```
"""Document-level tampering detection shared by app.py and evaluate.py.

A document is scanned with overlapping WINDOW x WINDOW windows at its
original
resolution - exactly how the training patches were made - and every window is
scored by the EfficientNetV2B0 patch model. The document score is the score
of
its most suspicious window. Grad-CAM is computed for the most suspicious
windows
and pasted back onto the full page.
"""

import json
import os

import numpy as np
import tensorflow as tf
from PIL import Image, ImageFilter

BASE_DIR = os.path.dirname(os.path.abspath(__file__))
MODEL_PATH = os.path.join(BASE_DIR, "models",
                           "efficientnetv2b0_patch_tampering_v2.keras")
THRESHOLD_PATH = os.path.join(BASE_DIR, "models", "document_threshold.json")

WINDOW = 224
STRIDE = 112
BATCH = 32
GRADCAM_WINDOWS = 3
MAX_SIDE = 2400          # very large photos are scaled down so analysis
                          # stays fast
MIN_PIXEL_STD = 12.0     # blank windows (empty paper) are not scored
# Training receipts are almost colourless (99 % of their windows have a mean
# saturation below 11). Strongly coloured windows - logos, colour graphics -
# are
# outside what the model has learned, so they are not scored.
MAX_WINDOW_SATURATION = 20.0
OUT_OF_DOMAIN_FRACTION = 0.05

class TamperDetector:
```

```

def __init__(self, model_path=None):
    # settings chosen by evaluate.py: model file, decision threshold,
    score aggregation
    saved = {}
    if os.path.exists(THRESHOLD_PATH):
        with open(THRESHOLD_PATH) as f:
            saved = json.load(f)
    if model_path is None:
        model_path = os.path.join(BASE_DIR, "models", saved["model"]) if
"model" in saved else MODEL_PATH
    self.model = tf.keras.models.load_model(model_path)
    same_model = os.path.basename(model_path) == saved.get("model",
os.path.basename(MODEL_PATH))
    self.threshold = float(saved.get("threshold", 0.5)) if same_model
else 0.5
    self.aggregation = saved.get("aggregation", "max") if same_model else
"max"
    self.backbone = self.model.get_layer("efficientnetv2-b0")
    names = [layer.name for layer in self.model.layers]
    split = names.index("efficientnetv2-b0")
    # layers before the backbone (augmentation, forensic filter) and
after it (classifier)
    self.pre = [l for l in self.model.layers[1:split] if l.name !=
"data_augmentation"]
    self.head = self.model.layers[split + 1:]
    # windows
    @staticmethod
    def prepare(image):
        image = image.convert("RGB")
        scale = min(1.0, MAX_SIDE / max(image.size))
        if scale < 1.0:
            image = image.resize((round(image.width * scale),
round(image.height * scale)), Image.LANCZOS)
        if image.width < WINDOW or image.height < WINDOW:
            canvas = Image.new("RGB", (max(image.width, WINDOW),
max(image.height, WINDOW)), (255, 255, 255))
            canvas.paste(image, (0, 0))
            image = canvas
        return image

    @staticmethod
    def positions(length):
        starts = list(range(0, length - WINDOW + 1, STRIDE))
        if starts[-1] != length - WINDOW:
            starts.append(length - WINDOW)
        return starts

    @staticmethod
    def model_input(image):
        """The training receipts are black-and-white scans, so every document
is
        analysed in grey-scale (3 identical channels) to match them."""
        grey = np.asarray(image.convert("L"), dtype=np.float32)
        return np.repeat(grey[:, :, None], 3, axis=2)

    def windows(self, image):
        array = self.model_input(image)
        grey = array[:, :, 0]
        saturation = np.asarray(image.convert("HSV"), dtype=np.float32)[:, :,
1]
        boxes, crops = [], []
        content = colour = 0
        for y in self.positions(image.height):
            for x in self.positions(image.width):
                if grey[y:y + WINDOW, x:x + WINDOW].std() < MIN_PIXEL_STD:
                    continue # blank paper
                content += 1

```

```

        if saturation[y:y + WINDOW, x:x + WINDOW].mean() >
MAX_WINDOW_SATURATION:
            colour += 1                                # coloured
            graphic, never seen in training
            continue
            boxes.append((x, y, x + WINDOW, y + WINDOW))
            crops.append(array[y:y + WINDOW, x:x + WINDOW])
        if not boxes:                                    # blank or fully
graphic image
            boxes.append((0, 0, WINDOW, WINDOW))
            crops.append(array[:WINDOW, :WINDOW])
            self.last_colour_fraction = colour / max(content, 1)
            return boxes, np.stack(crops)

    def score(self, image):
        """Returns the prepared image, window boxes and window
probabilities."""
        image = self.prepare(image)
        boxes, crops = self.windows(image)
        probs = self.model.predict(crops, batch_size=BATCH, verbose=0)[: , 0]
        return image, boxes, probs

    @staticmethod
    def aggregate(probs, method):
        """Combines the window scores of one document into a document
score."""
        ranked = np.sort(probs)[::-1]
        if method == "max":
            return float(ranked[0])
        if method.startswith("top"):
            return float(ranked[:int(method[3:])] .mean())
        return float(ranked.mean())

    # Grad-CAM
    def gradcam(self, crop):
        batch = tf.convert_to_tensor(crop[None, ...], dtype=tf.float32)
        for layer in self.pre:
            batch = layer(batch, training=False)
        features = self.backbone(batch, training=False)          # 7 x 7 x 1280
        with tf.GradientTape() as tape:
            tape.watch(features)
            x = features
            for layer in self.head:
                x = layer(x, training=False)
            score = x[:, 0]
        grads = tape.gradient(score, features)
        weights = tf.reduce_mean(grads, axis=(1, 2))[0]
        cam = tf.nn.relu(tf.reduce_sum(features[0] * weights, axis=-
1)).numpy()
        cam = cam / (cam.max() + 1e-8)
        return np.asarray(Image.fromarray(np.uint8(cam *
255)).resize((WINDOW, WINDOW), Image.BICUBIC),
                    dtype=np.float32) / 255.0

    # full analysis used by the web application
    def analyze(self, image):
        image, boxes, probs = self.score(image)
        best = int(np.argmax(probs))
        doc_prob = self.aggregate(probs, self.aggregation)
        label = "Tampered" if doc_prob >= self.threshold else "Genuine"

        # Grad-CAM for the most suspicious windows, weighted by their score
        array = self.model_input(image)
        cam_map = np.zeros(array.shape[:2], dtype=np.float32)
        order = np.argsort(probs)[::-1][:GRADCAM_WINDOWS]
        for i in order:
            if i != best and probs[i] < probs[best] * 0.8:
                continue
            x1, y1, x2, y2 = boxes[i]
            cam = self.gradcam(array[y1:y2, x1:x2]) * float(probs[i])

```

```

        cam_map[y1:y2, x1:x2] = np.maximum(cam_map[y1:y2, x1:x2], cam)
    if cam_map.max() > 0:
        # soften the square window borders so the map reads as one
        continuous heatmap
        blurred = Image.fromarray(np.uint8(cam_map / cam_map.max() *
255)).filter(ImageFilter.GaussianBlur(12))
        cam_map = np.asarray(blurred, dtype=np.float32)
        cam_map /= cam_map.max() + 1e-8

    return {
        "image": image,
        "label": label,
        "probability": doc_prob,
        "threshold": self.threshold,
        "windows": len(boxes),
        "flagged": int((probs >= self.threshold).sum()),
        "average": float(probs.mean()),
        "boxes": boxes,
        "probs": probs,
        "best_box": boxes[best],
        "cam": cam_map,
        "colour_fraction": self.last_colour_fraction,
        "out_of_domain": self.last_colour_fraction >=
OUT_OF_DOMAIN_FRACTION,
    }

```

5.2 SCREENSHOT OF THE APPLICATION

The following screenshots were taken from the running FinDoc AI web application. The navigation bar at the top of every page gives access to the Home, Detect, How It Works, Performance and About pages.

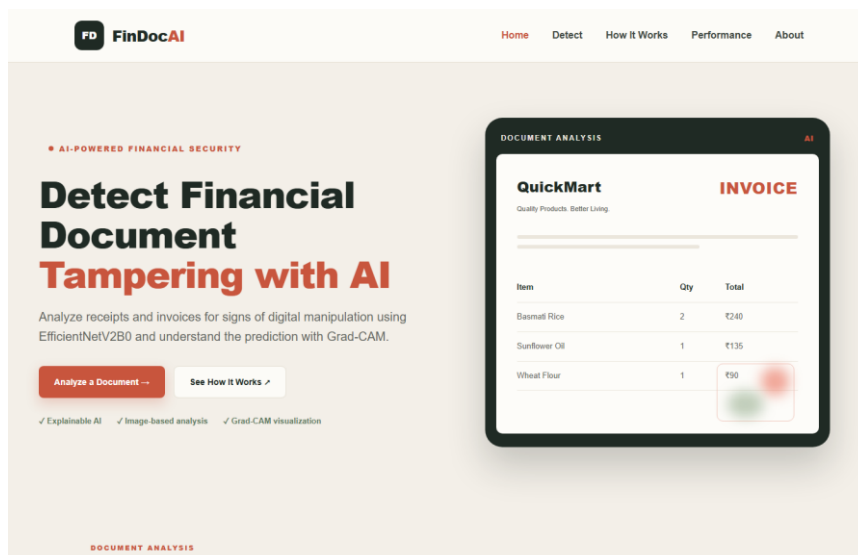


Figure 5.1: Home Page of FinDoc AI

The Home page introduces the system with the headline “Detect Financial Document Tampering with AI” and buttons that open the Detect page and the How It Works page.

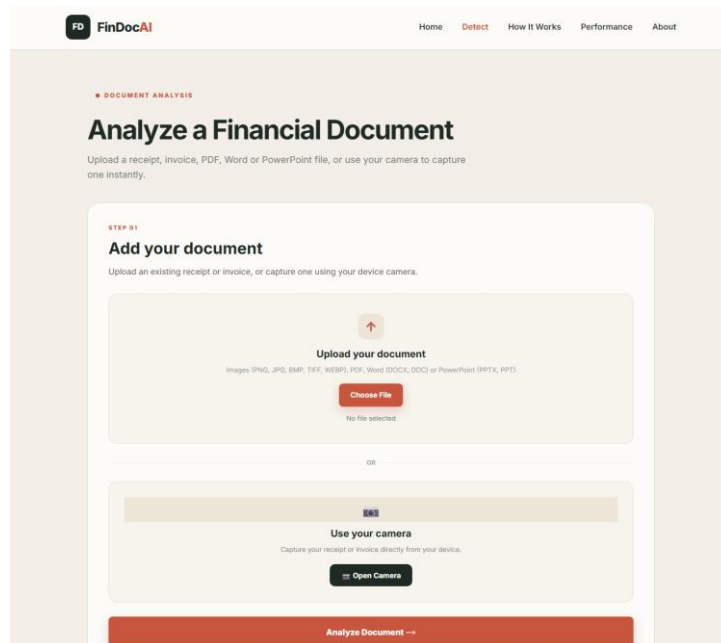


Figure 5.2: Detect Page – Upload a File or Use the Camera

On the Detect page the user either chooses a file – an image, a PDF or a Word document – or opens the device camera and captures the document, and then clicks Analyze Document.

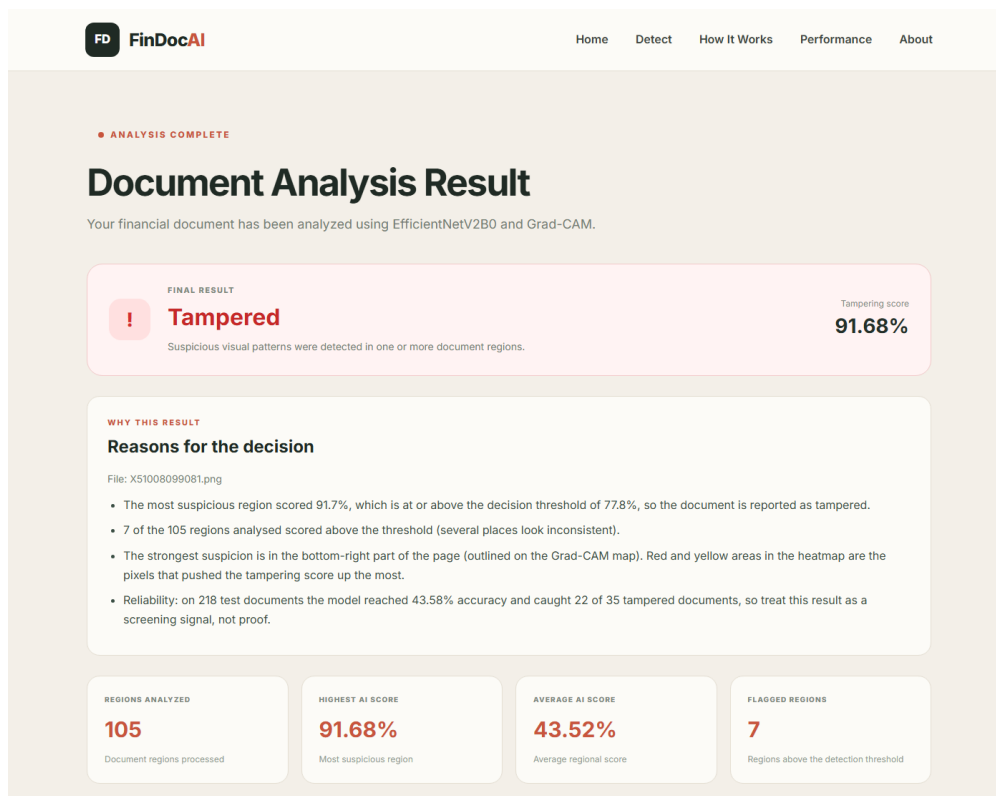


Figure 5.3: Result Page for a Tampered Receipt – Verdict and Reasons

Figure 5.3 shows the result for test receipt X51008099081, which is tampered and was correctly detected. The verdict card reports the tampering score of the most suspicious region, and the “Why this result” section explains in plain language why the document was classified this way: the score compared with the 77.85% decision threshold, how many regions were flagged and where the strongest suspicion lies.

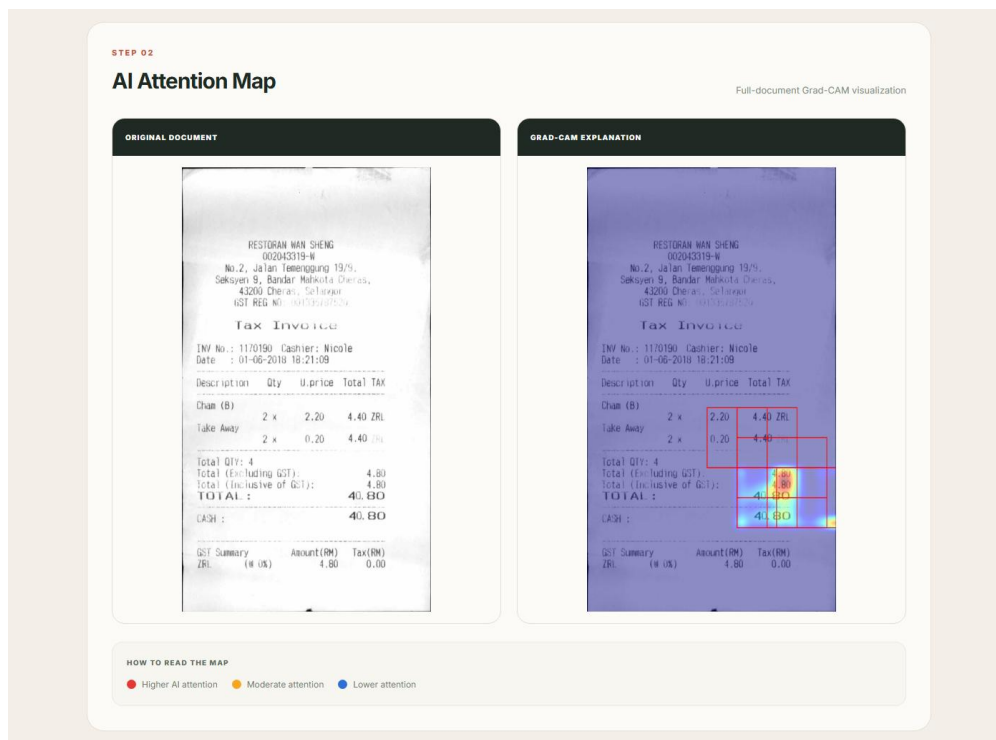


Figure 5.4: Result Page – Original Document and Full-page Grad-CAM Map

The attention map shows the original page next to the Grad-CAM overlay. Regions above the threshold are outlined in red, and red and yellow areas mark the pixels that raised the tampering score. For this receipt one of the flagged regions covers an edited price.

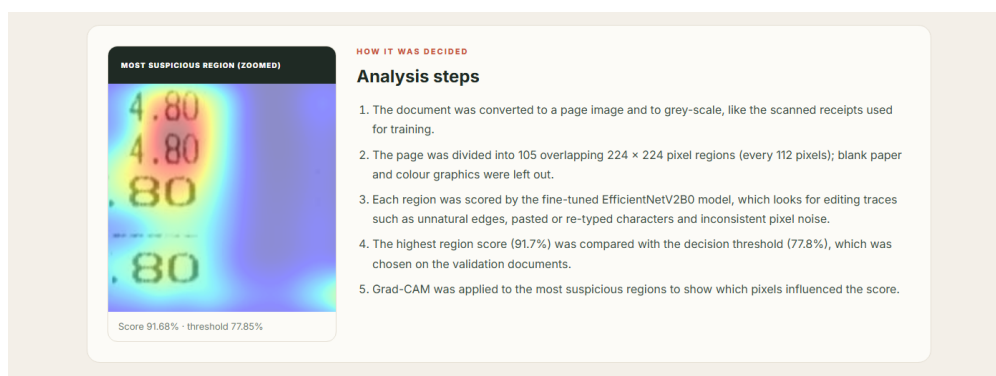


Figure 5.5: Result Page – Zoomed Suspicious Region and Analysis Steps

Below the map, the most suspicious region is shown enlarged together with its heatmap, and the “How it was decided” list gives the exact steps and numbers used for this document.

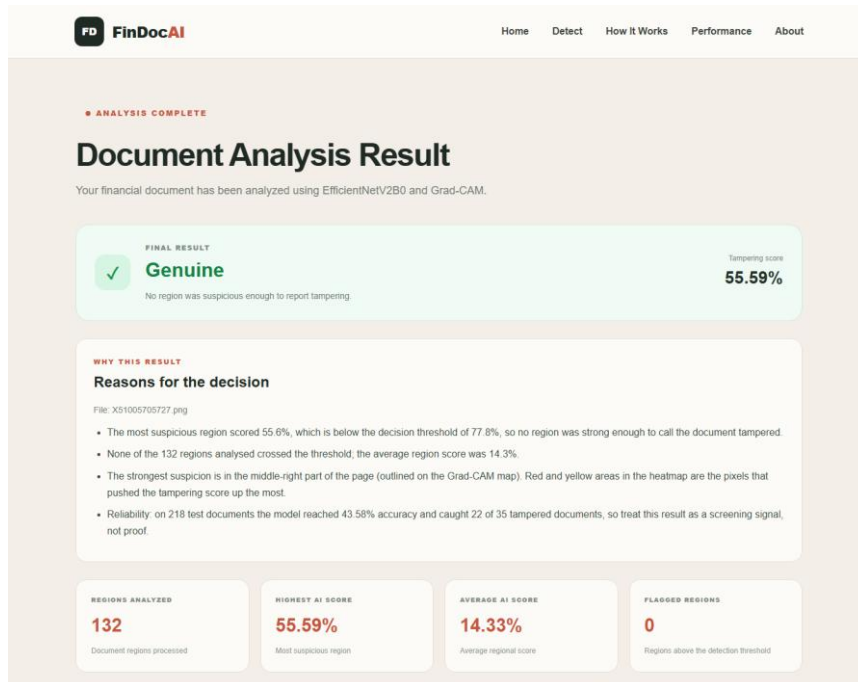


Figure 5.6: Result Page for a Genuine Receipt

For a genuine receipt no region crosses the threshold, and the reasons state the highest and average region scores.

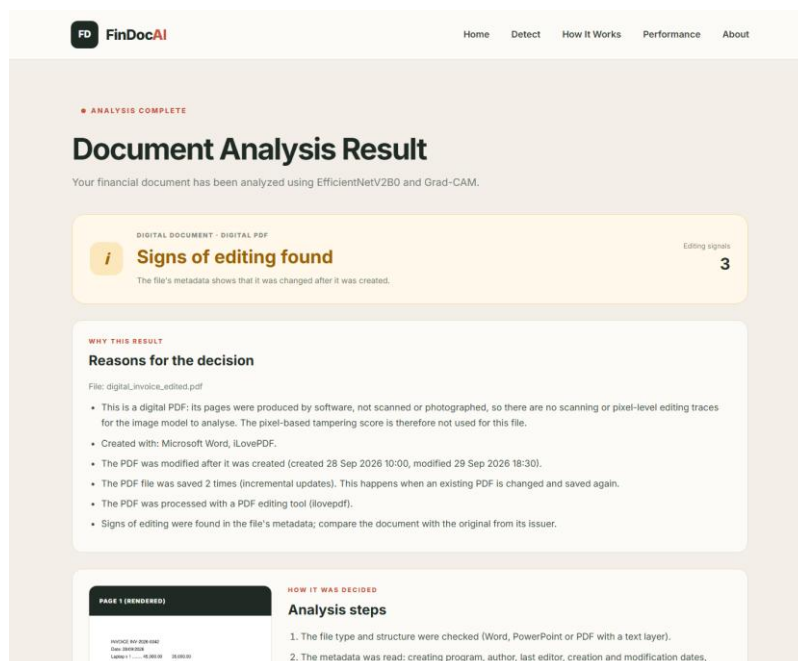


Figure 5.7: Result Page for an Edited Digital PDF (Metadata Check)

Figure 5.7 shows the result for a digital invoice PDF whose amount was changed and which was saved again with a PDF editing tool: the metadata check reports three editing signals and lists each of them.

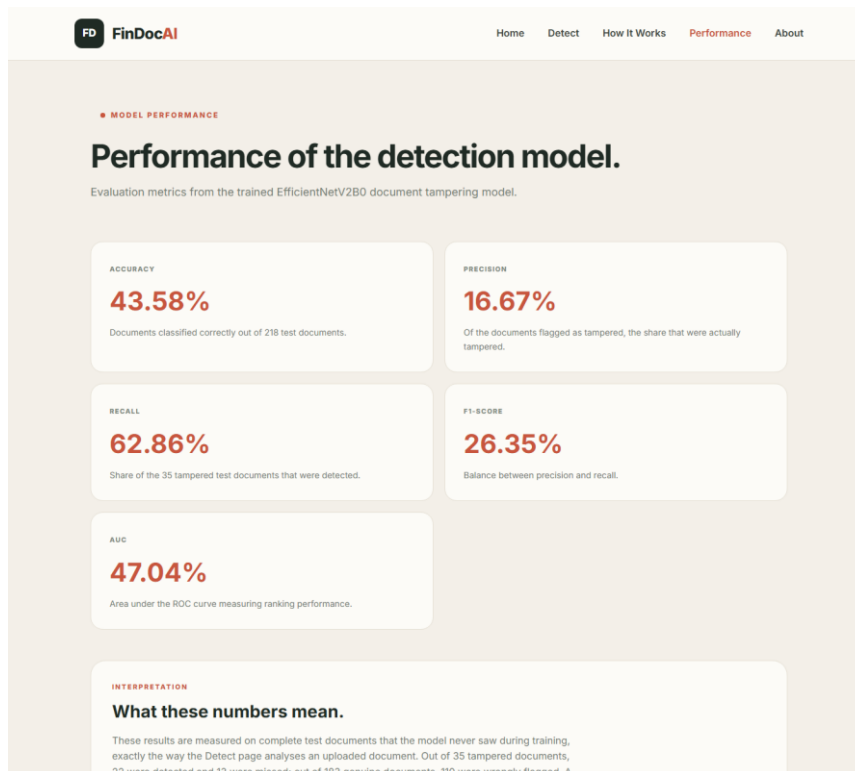


Figure 5.8: Performance Page Showing the Measured Test Results

The Performance page reads the measured document-level results from the evaluation script, so the numbers shown to users are always the real test results discussed in Chapter 6.

CHAPTER-6

CONCLUSION

Evaluation Metrics

The system is evaluated at document level on the 218 test receipts (35 tampered, 183 genuine), exactly the way the web application analyses an uploaded document. None of these receipts was used for training or for choosing the threshold. With Tampered as the positive class, TP, TN, FP and FN denote true positives, true negatives, false positives and false negatives. The following metrics are used:

Accuracy = $(TP + TN) / (TP + TN + FP + FN)$ – the fraction of all documents classified correctly.

Precision = $TP / (TP + FP)$ – of the documents flagged as tampered, the fraction that are actually tampered.

Recall = $TP / (TP + FN)$ – of the tampered documents, the fraction that the system detects.

F1-score = $2 \times \text{Precision} \times \text{Recall} / (\text{Precision} + \text{Recall})$ – the harmonic mean of precision and recall.

ROC-AUC – the area under the ROC curve; 1.0 is a perfect ranking of tampered above genuine documents and 0.5 is the level of random guessing.

Because only one receipt in six is tampered, accuracy alone is misleading: a system that labels every document as genuine would reach 84% accuracy while detecting nothing. Precision, recall, F1-score and above all ROC-AUC therefore give the more reliable picture.

Results

Training results. Figure 6.1 shows the training and validation accuracy and loss for both training phases (the dashed line marks the start of fine-tuning). Training

accuracy keeps rising while validation accuracy stays close to 60%, which shows that the model learns the training patches but generalises only weakly to new receipts.

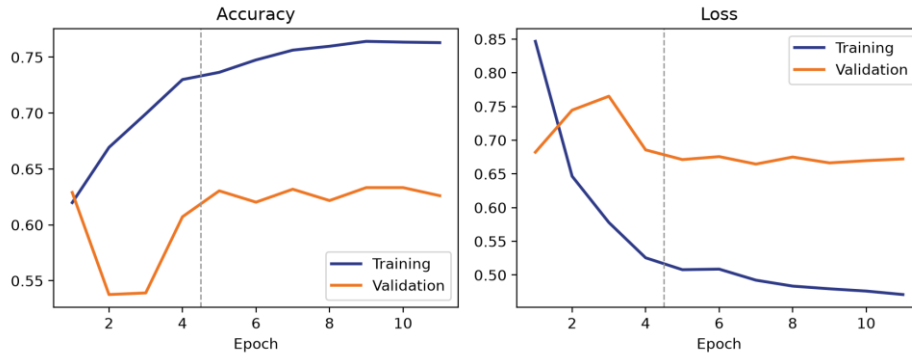


Figure 6.1: Training and Validation Accuracy and Loss Curves

On the 786 test patches, the model reaches an accuracy of 67.56 %, precision 38.91 %, recall 46.04 % and AUC 63.55 %.

Document-level results. Table 6.1 summarises the results on the complete test documents with the decision threshold of 77.85% chosen on the validation documents. These are the numbers shown on the Performance page of the application (Figure 5.8).

Table 6.1: Document-level Results on the Test Receipts

Metric	Value
Accuracy	43.58 %
Precision (Tampered)	16.67 %
Recall (Tampered)	62.86 %
F1-score (Tampered)	26.35 %
ROC-AUC	47.04 %
Tampered receipts detected	22 of 35
Genuine receipts wrongly flagged	110 of 183

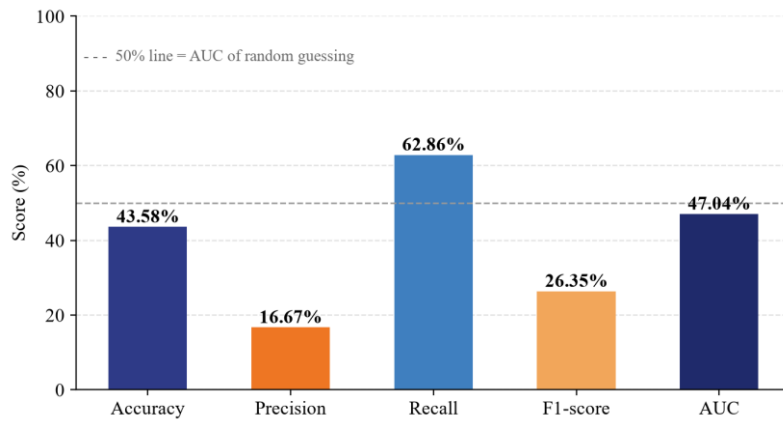


Figure 6.2: Document-level Evaluation Metrics

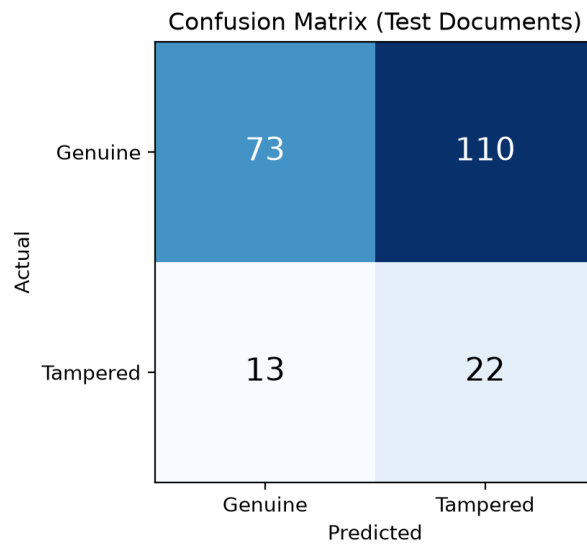


Figure 6.3: Confusion Matrix on the Test Receipts

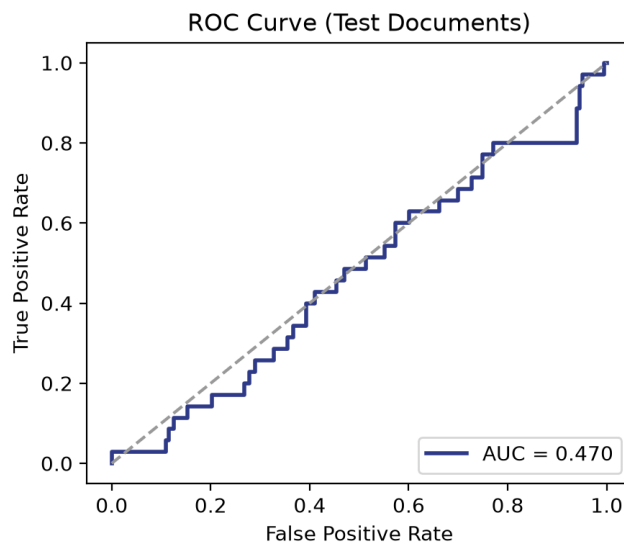


Figure 6.4: ROC Curve on the Test Receipts

The confusion matrix (Figure 6.3) shows that 22 of the 35 tampered receipts were detected and 13 were missed, while 73 of the 183 genuine receipts were correctly passed and 110 were wrongly flagged. The ROC-AUC of 47.04 % (Figure 6.4) is close to 50%, which means that, at document level, the tampering score does not yet separate tampered from genuine receipts better than chance. The patch-level AUC of 63.55 % shows that the model does learn something about individual edited areas, but a receipt contains 20 to 60 windows and, with this level of accuracy, almost every genuine receipt also has one window with a high score.

Analysis of the initial model. The first version of the project reported 97.85% accuracy, 91.43% precision, 95.05% recall and 97.80% AUC on its test patches. When that model was tested on complete receipts in the way the application uses it, it reached only 49.5% accuracy and an AUC of 0.44. A diagnosis showed the reason: its tampered patches were small crops enlarged about three times, while its genuine patches were larger crops that were reduced. Small crops taken from genuine receipts were classified as tampered half of the time, and 256×256 windows that contained real edits received an average tampering score of only 11.7%. The model had learned the zoom level of the patches rather than tampering. Table 6.2 compares the two versions.

Table 6.2: Initial Model versus Final Model

Measure	Initial model	Final model
Tampered / genuine patch size	$\approx 80 \times 85$ px (enlarged) / 256×256 px (reduced)	224×224 px / 224×224 px (original scale)
Reported patch accuracy	97.85 %	67.56 %
Document-level accuracy	49.5 %	43.58 %
Document-level AUC	44.3 %	47.04 %
Genuine receipts wrongly flagged	89 of 183	110 of 183
Tampered receipts detected	14 of 35	22 of 35

Neither version separates tampered from genuine receipts reliably. The final model detects more of the tampered receipts (22 instead of 14) but also flags more genuine receipts (110 instead of 89). The important difference is that its evaluation is honest:

its patches, its training and its document-level test all use the same scale as the real application, so the reported numbers describe what a user of the web application will actually experience.

Grad-CAM Analysis

Grad-CAM makes the model's behaviour visible, which is especially valuable when accuracy is limited. For receipt X51008099081 (Figures 5.3 to 5.5) one of the flagged regions covers an edited price, so the explanation points the reviewer to the right place. For most of the other correctly detected receipts, however, the flagged regions did not cover the annotated edits: the documents were flagged because of other areas such as bold totals, stamps or dense text. Grad-CAM exposes these cases immediately, so a reviewer can see when the model is right for the wrong reason instead of trusting the label blindly.

The explanations also revealed a domain limitation. A colourful hospital receipt template was first flagged as tampered because of its coloured logo and wave graphic, which never occur in the black-and-white training receipts. Strongly coloured windows are now skipped and such documents receive a warning on the result page, but documents that differ strongly from scanned retail receipts should be treated with caution.

Testing

Testing was carried out at several levels to make sure that each part of the system works correctly on its own and together with the others.

Unit testing: the window generation, the prediction, the score aggregation, the Grad-CAM computation and the file conversion functions were tested with sample receipts, PDF files and Word documents to verify shapes, value ranges and labels.

Integration testing: the Flask routes were tested with the Flask test client, uploading images, single- and multi-page PDF files, Word documents with and without pictures, invalid files and empty requests.

System testing: the complete application was run and used in the browser, from the Home page through upload and camera capture to the result, Performance and About pages.

The functional test cases and their results are listed in Table 6.3.

Table 6.3: Functional Test Cases

ID	Test Case	Expected Result	Status
TC1	Open all five pages	Home, Detect, How It Works, Performance and About load	Pass
TC2	Upload a genuine receipt image	Result with score, threshold, reasons and Grad-CAM map	Pass
TC3	Upload a tampered receipt image	Result with score, flagged regions outlined and reasons	Pass
TC4	Capture a document with Use Camera	Captured photo is analysed like an uploaded file	Pass
TC5	Upload a single-page and a two-page PDF	Pages rendered; most suspicious page shown	Pass
TC6	Upload a Word / PowerPoint file with a pasted receipt	Embedded picture extracted and analysed	Pass
TC7	Upload a typed Word, PowerPoint or digital PDF	Recognised as digital; metadata assessment shown	Pass
TC8	Upload a digital PDF edited and re-saved with iLovePDF	Signs of editing found (3 signals)	Pass
TC9	Upload a text file or corrupted image	Clear error message; application keeps running	Pass
TC10	Upload a colourful document template	Colour graphics skipped; document-type warning shown	Pass

Conclusion

In this project, an explainable financial document tampering detection system, FinDoc AI, was designed, implemented and evaluated on the public “Find it again!” receipt forgery dataset. The system fine-tunes EfficientNetV2B0 on 224×224 windows cut from the receipts at their original resolution, scans complete documents with overlapping windows, and explains every decision with Grad-CAM heatmaps, flagged regions and a plain-language list of reasons. The

interactive Flask web application accepts images, camera photos, PDF, Word and PowerPoint files, assesses typed digital files through their metadata, and runs on an ordinary laptop.

A key result of the project is methodological. The initial model appeared to be highly accurate (97.85%), but a careful document-level evaluation showed that it had learned the size of its training patches rather than tampering. After correcting the patch preparation, the final system reaches 43.58 % accuracy and a document-level AUC of 47.04 % on the test receipts. Detecting careful, often single-character manual edits in scanned receipts with an image-only CNN trained on 94 tampered receipts therefore remains an open problem. The value of the delivered system lies in its complete, honest pipeline and its explanations, which show a human reviewer where to look; it should be used as a screening aid, not as proof of tampering.

Limitations and Future Scope

The evaluation points to clear directions for future work:

More tampered training data: with only 94 tampered receipts for training the model over-fits; synthetic forgeries (automatically copy-moving and re-typing digits) could provide thousands of additional examples.

Forensic features: noise-residual filters were tried in this project without a clear gain; dedicated forensic networks trained with pixel-level masks may capture editing traces better than an ImageNet-pre-trained classifier.

Text-based consistency checks: reading the receipt with OCR and checking that item prices, totals and tax add up would catch many forgeries that leave no visible pixel traces.

More document types: bank statements, invoices, cheques and digital PDFs need their own training data before the system can be relied on for them.

6.1 REFERENCES

- [1] M. Tan and Q. V. Le, “EfficientNetV2: Smaller Models and Faster Training,” in Proc. 38th International Conference on Machine Learning (ICML), 2021, pp. 10096–10106.
- [2] M. Tan and Q. V. Le, “EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks,” in Proc. 36th International Conference on Machine Learning (ICML), 2019, pp. 6105–6114.
- [3] R. R. Selvaraju, M. Cogswell, A. Das, R. Vedantam, D. Parikh and D. Batra, “Grad-CAM: Visual Explanations from Deep Networks via Gradient-based Localization,” in Proc. IEEE International Conference on Computer Vision (ICCV), 2017, pp. 618–626.
- [4] K. He, X. Zhang, S. Ren and J. Sun, “Deep Residual Learning for Image Recognition,” in Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2016, pp. 770–778.
- [5] K. Simonyan and A. Zisserman, “Very Deep Convolutional Networks for Large-Scale Image Recognition,” in Proc. International Conference on Learning Representations (ICLR), 2015.
- [6] A. Krizhevsky, I. Sutskever and G. E. Hinton, “ImageNet Classification with Deep Convolutional Neural Networks,” in Advances in Neural Information Processing Systems (NeurIPS), 2012.
- [7] B. M. Tornés, T. Taburet, E. Boros, K. Rouis, A. Doucet, P. Gomez-Krämer, N. Sidère and V. Poulain d’Andecy, “Receipt Dataset for Document Forgery Detection,” in Proc. International Conference on Document Analysis and Recognition (ICDAR), 2023.
- [8] C. Artaud, N. Sidère, A. Doucet, J.-M. Ogier and V. Poulain d’Andecy, “Find it! Fraud Detection Contest Report,” in Proc. 24th International Conference on Pattern Recognition (ICPR), 2018, pp. 13–18.
- [9] P. Zhou, X. Han, V. I. Morariu and L. S. Davis, “Learning Rich Features for Image Manipulation Detection,” in Proc. IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2018, pp. 1053–1061.
- [10] Y. Wu, W. AbdAlmageed and P. Natarajan, “ManTra-Net: Manipulation Tracing Network for Detection and Localization of Image Forgeries with Anomalous Features,” in Proc. IEEE/CVF CVPR, 2019, pp. 9543–9552.
- [11] B. Bayar and M. C. Stamm, “A Deep Learning Approach to Universal Image Manipulation Detection Using a New Convolutional Layer,” in Proc. 4th ACM Workshop on Information Hiding and Multimedia Security, 2016, pp. 5–10.
- [12] J. Fridrich and J. Kodovsky, “Rich Models for Steganalysis of Digital Images,” IEEE Transactions on Information Forensics and Security, vol. 7, no. 3, pp. 868–882, 2012.
- [13] N. Krawetz, “A Picture’s Worth: Digital Image Analysis and Forensics,” Black Hat Briefings, 2007.

- [14] A. Dosovitskiy et al., “An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale,” in Proc. International Conference on Learning Representations (ICLR), 2021.
- [15] I. Goodfellow, Y. Bengio and A. Courville, Deep Learning. MIT Press, 2016.
- [16] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li and L. Fei-Fei, “ImageNet: A Large-Scale Hierarchical Image Database,” in Proc. IEEE CVPR, 2009, pp. 248–255.
- [17] B. Zhou, A. Khosla, A. Lapedriza, A. Oliva and A. Torralba, “Learning Deep Features for Discriminative Localization,” in Proc. IEEE CVPR, 2016, pp. 2921–2929.
- [18] A. Chattopadhyay, A. Sarkar, P. Howlader and V. N. Balasubramanian, “Grad-CAM++: Generalized Gradient-Based Visual Explanations for Deep Convolutional Networks,” in Proc. IEEE Winter Conference on Applications of Computer Vision (WACV), 2018, pp. 839–847.
- [19] D. P. Kingma and J. Ba, “Adam: A Method for Stochastic Optimization,” in Proc. International Conference on Learning Representations (ICLR), 2015.
- [20] Artifex Software, “PyMuPDF Documentation,” <https://pymupdf.readthedocs.io>
- [21] A. W. Harley, A. Ufkes and K. G. Derpanis, “Evaluation of Deep Convolutional Nets for Document Image Classification and Retrieval,” in Proc. International Conference on Document Analysis and Recognition (ICDAR), 2015, pp. 991–995.
- [22] F. Cruz, N. Sidère, M. Coustaty, V. Poulain d’Andecy and J.-M. Ogier, “Local Binary Patterns for Document Forgery Detection,” in Proc. 14th IAPR International Conference on Document Analysis and Recognition (ICDAR), 2017, pp. 1223–1228.
- [23] TensorFlow Developers, “TensorFlow and Keras Documentation – EfficientNetV2B0,” https://www.tensorflow.org/api_docs/python/tf/keras/applications/EfficientNetV2B0
- [24] Pallets Projects, “Flask Documentation,” <https://flask.palletsprojects.com>

6.2 APPENDIX : SOURCE CODE

This appendix contains the main part of the Flask application (app.py) – the configuration, model loading and the predict route – followed by the installation steps and the list of required libraries. The remaining parts of app.py render the other pages, draw the Grad-CAM images and build the list of reasons.

A. Flask Application (app.py – main part)

```
# app.py (main part) - configuration, model loading and the predict route
import json
import os
import uuid

import matplotlib
matplotlib.use("Agg")
import matplotlib.patches as mpatches
import matplotlib.pyplot as plt
import numpy as np
from flask import Flask, render_template, request
from werkzeug.utils import secure_filename

from converter import ALLOWED_EXTENSIONS, ConversionError, extension,
load_pages
from detector import TamperDetector, WINDOW, STRIDE

# ---- CONFIGURATION ----

BASE_DIR = os.path.dirname(os.path.abspath(__file__))
UPLOAD_FOLDER = os.path.join(BASE_DIR, "static", "uploads")
RESULT_FOLDER = os.path.join(BASE_DIR, "static", "results")
METRICS_PATH = os.path.join(BASE_DIR, "models", "document_metrics.json")

os.makedirs(UPLOAD_FOLDER, exist_ok=True)
os.makedirs(RESULT_FOLDER, exist_ok=True)

# ---- FLASK ----

app = Flask(__name__)
app.config["UPLOAD_FOLDER"] = UPLOAD_FOLDER
app.config["MAX_CONTENT_LENGTH"] = 25 * 1024 * 1024      # 25 MB upload limit

# ---- LOAD MODEL ----

print("=" * 60)
print("Loading EfficientNetV2B0 model...")
print("=" * 60)
detector = TamperDetector()
print(f"Model loaded. Document threshold: {detector.threshold:.3f}")

# ---- PREDICT ----

@app.route("/predict", methods=["POST"])
def predict():
    file = request.files.get("document")
    if file is None or file.filename == "":
```

```

        return render_template("detect.html", error="Please upload or capture
a document.")
    if extension(file.filename) not in ALLOWED_EXTENSIONS:
        return render_template("detect.html",
                                error="Supported files: images (PNG, JPG, BMP,
TIFF, WEBP), PDF and Word (DOCX, DOC).")

    try:
        pages = load_pages(file.read(), file.filename)
    except ConversionError as e:
        return render_template("detect.html", error=str(e))

    # analyse every page and keep the most suspicious one
    results = [(label, detector.analyze(image)) for label, image in pages]
    page_label, analysis = max(results, key=lambda r: r[1]["probability"])
    if any(r["label"] == "Tampered" for _, r in results):
        analysis["label"] = "Tampered"

    stem = os.path.splitext(secure_filename(file.filename))[0] or "document"
    unique_name = uuid.uuid4().hex + "_" + stem + ".png"
    analysis["image"].save(os.path.join(UPLOAD_FOLDER, unique_name))

    try:
        gradcam_url = create_gradcam_image(analysis)
        region_url = create_region_image(analysis)
    except Exception as e:
        # the verdict is still shown
        without the maps
        print("Grad-CAM error:", e)
        gradcam_url = region_url = None

    reasons, steps = explain(analysis, page_label, len(pages),
load_metrics())
    probability = analysis["probability"]

    return render_template(
        "result.html",
        result=analysis["label"],
        confidence=round(probability * 100, 2),
        highest_score=round(probability * 100, 2),
        average_score=round(analysis["average"] * 100, 2),
        regions=analysis["windows"],
        flagged_regions=analysis["flagged"],
        threshold=round(analysis["threshold"] * 100, 2),
        out_of_domain=analysis["out_of_domain"],
        file_name=file.filename,
        page_label=page_label if len(pages) > 1 else None,
        page_count=len(pages),
        reasons=reasons,
        steps=steps,
        original_image="/static/uploads/" + unique_name,
        gradcam_image=gradcam_url,
        region_image=region_url,
    )

```

B. Installation and Execution Steps

Step 1: Install Python 3.10 or above (the project was tested with Python 3.13 on Windows 11).

Step 2: Open a terminal in the project folder and create a virtual environment:
`python -m venv venv`

Step 3: Install the dependencies: `venv\Scripts\python -m pip install -r requirements.txt`

Step 4: Start the website: `venv\Scripts\python app.py` and open `http://127.0.0.1:5000` in a browser.

Step 5: Optional retraining: run `prepare_patches.py`, then `patch_train.py`, then `evaluate.py` with the same Python; the new model, threshold and metrics are picked up automatically.

C. requirements.txt

```
tensorflow==2.20.*
keras==3.12.*
flask>=3.0
numpy
pandas
pillow
scikit-learn
matplotlib
pymupdf
pywin32
```